



UNIVERSITY OF
CAMBRIDGE

Department of Computer
Science and Technology

May 15, 2026

Efficient Coinductives through State-Machine Corecursors

William Sørensen

Gonville & Caius College

Submitted in partial fulfilment of the requirements for the
Computer Science Tripos, Part II

Declaration

I, William Sørensen of Gonville & Caius College, being a candidate for the course, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose.

Signed: William Sørensen

Date: May 15, 2026

Proforma

Word count 9320

Line count 8701 + PR

Year 2026

Project originator The candidate

Day-to-day supervisor Alex Keizer, Tobias Grosser

Marking supervisor Jamie Vicary

Ethics approval N / A

Project Aims

Work completed

Special difficulties

My Supervisor had to take a break half way through the project.

My grandfather Stein-Ulf Sørensen Passed away (2025-10-28), my aunt Asbjørg Johanne Raanes Passed away (2025-12-19).

Acknowledgements

Contents

Project Aims	iii
Work completed	iii
Special difficulties	iii
1 Introduction	1
2 Preparation	2
2.1 Dependent type theory	2
2.2 Lean	2
2.3 Polynomial functors	4
2.3.1 Common polynomials	4
2.4 Recursion and Inductive datatype	5
2.5 Corecursion and coinductive types	5
2.5.1 In OCaml	6
2.5.2 Streams	6
2.5.3 The M -type	6
2.5.4 State-Machine encoding	8
2.6 Bisimilarity	8
2.7 The Free Monad	8
2.8 Interaction trees	9
2.9 Methodology	10
2.10 Tools used	10
2.11 Requirement Analysis	10
2.11.1 State-machine encoding (Core)	10
2.11.2 Non-termination-monad (Core)	11
2.11.3 AltRepr Type (Extension)	11
2.11.4 Interaction Trees (Extension)	11
2.11.5 Free Monad (Extension)	11
3 Implementation	12
3.1 Common	12
3.1.1 Transliteration	13
3.1.2 Expanding the progressive approximation theory	13
3.1.3 Polynomial equivalents	14
3.1.4 Natural isomorphisms of MvFunctors	14
3.2 Stream implementation	14
3.2.1 State-machine streams	15
3.2.2 Progressive approximation streams	16
3.2.3 Proving the equivalence	16
3.3 State-machine encoding of M -types	17
3.3.1 Rephrasing bisimilarity for M -types	17
3.3.2 PreM	17
3.3.3 SM	19
3.3.4 Proving the equivalence	19
3.4 The Alternative Representation (AltRepr) Type	20
3.4.1 High performance Low universe M -type (HpLuM)	21
3.5 Non-Termination Monad (NTMonad)	22
3.6 Interaction Trees	22
3.6.1 The ITree Monad	23

3.6.2	Weak bisimilarity	24
3.6.3	A formally verified optimization	25
3.7	Free Monad	25
4	Evaluation	28
4.1	State-machine encoding	28
4.1.1	Performance between SME and PA	29
4.1.2	Comparison of corecursor implementations	29
4.2	AltRepr Type	30
4.3	Interaction trees	30
4.4	Non Termination Monad	31
4.5	Futumorphic productivity	31
4.5.1	Comparing function definitions	31
5	Conclusions	33
5.1	Future work	33
5.1.1	Syntax macro for types	33
5.1.2	Derecursifier for corecursive function	33
5.1.3	Extending the ITree library	33
	Bibliography	34
6	Appendices	36
6.A	Project import graph	37
6.B	Inductive predicate	37
6.C	Coinductive predicate	37
6.D	Unsoundness of old shrink definition	37
7	Proposal	39

List of figures

Figure 1	Inference rules for Π and Σ types	2
Figure 2	Curry-Howard correspondance	2
Figure 3	Some strong-induction primitives from Lean	3
Figure 4	Gcd as Source, Kernel and LCNF	3
Figure 5	Formal definition of polynomial functor	4
Figure 6	Projection polynomial	4
Figure 7	Constant polynomial	4
Figure 8	Product polynomial	5
Figure 9	Sum polynomial	5
Listing 1	Definition of list in OCaml	5
Listing 2	Definition of colist in OCaml	6
Figure 10	Agreement of two trees of height 1 and 2	7
Listing 3	Definition of approx for stream	7
Listing 4	Objects of the corecursor for streams	7
Listing 5	Destructor for streams	7
Figure 11	Two bisimilar state-machines	8
Listing 6	Bisimilarity on streams	8
Figure 12	Choice points for a producing a stream	9
Listing 7	Rocq defintion from [1]	9
Figure 13	Birds-eye view of components of the project	12
Listing 8	Transliteration between universes of types	13
Listing 9	Natural isomorphisms on MvFunctors	14
Listing 10	PreStream and destructors	15
Figure 14	PreStream setoid	15
Figure 15	corec and destructors for SStreams	15
Figure 16	Coinduction principle for SStreams	16
Figure 17	PStream definition	16
Figure 18	PStream destructors	16
Figure 19	Coinduction principle for PStreams	16
Listing 11	corec for PStreams	16
Figure 20	Key code involved in Stream equivlance	17
Figure 21	Equivalent bisimilarity definitions	17
Listing 12	PreM definition	18
Figure 22	Destructor and β -rule for PreM	18
Listing 13	ULifting of PreM	18
Listing 14	Bisimilarity of PreM types	18
Figure 23	Equivalence relation for PreM bisimilarity.	18
Listing 15	Bisimilarity of SM-types	19
Listing 16	Functions of the equivalence	19
Listing 17	SME and PA equivlance equivP	20
Listing 18	Mathlib [2] definition of Shrink	20
Figure 24	Universe transforming types.	21
Figure 25	Defintions of ITrees	22
Table 1	Functions implemented in the dissertaton (L)ean, in (R)ocq [1] and (M). Sammler [3]	23
Table 2	Equations implemented in the dissertaton (L)ean, in (R)ocq [1] and (M). Sammler [3] ¹	23

Figure 26	Visual representation of <code>bind</code> on <code>ITrees</code>	24
Figure 27	Weak-bisimilarity for <code>ITrees</code>	24
Listing 19	<code>Imp</code> and <code>simpl</code>	25
Figure 28	Relationship between <code>Free</code> monad and <code>DT</code>	26
Table 3	Functions implemented for the <code>Free</code> monad ²	27
Table 4	Requirements and completion	28
Figure 29	Performance of <code>PA</code> , <code>HpLuM</code> , and <code>SM & PreM</code> representation	29
Table 5		30
Listing 20	<code>LCNF</code> for functions on the <code>AltRepr</code> Type	30
Listing 21	Futomorphic unfolding lemma	31
Listing 22	Interlace constant	32
Listing 23	Stutter constant	32
Listing 24	Run-length encoding	32
Listing Y		37
Figure 30	Import graph of the lean component of the project as of 2026-04-04	37
Figure 31	Evenness inductive predicate, from first principles	37

¹The annotations in the table are as follows:

* = strong bisimilarity rather than equality in `Rocq`

† = proven using coinduction-up-to in `Rocq`

²The annotations in the table are as follows:

* = Cross universe version also implemented.

Chapter 1: Introduction

Lean [4] is a pure and total functional language, meaning all functions must terminate. Problems such as program simulation, or production of streams of data, often requires leaving the finite. It would limit expressivity requiring these to terminate. We call data like this *coinductive*. Researchers working on program verification are also interested in this, they use structures called interaction trees as a denotational semantic [1].

Lean has an implementation of coinductives [2], [5], [6]. These are implemented as a series of progressive approximations. We will refer to this as the PA encoding. As a limitation of this encoding unfolding a layer of a coinductive, takes time proportional to the depth of the layer. A consequence of this is getting a stream to depth n takes $\mathcal{O}(n)$ time. This gets dramatically worse as you map streams, becoming intractable for most programs. An alternative encoding stores a generating function, parameterized by some carrier, and an initial state. This is what I will refer to as the state-machine encoding. By construction, destructuring will be $\mathcal{O}(1)$. The downside of this definition is that it is impredicative, and therefore becomes harder to use and reason about.

This dissertation will implement the SME, prove it is equivalent to the PA encoding, and use this to construct an efficient coinduction library. Additionally we will implement interaction trees, with an equivalence relation that can be used to find contextually equivalent programs.

Chapter 2: Preparation

In this section I will discuss prerequisites for the implementation of the dissertation. I will discuss the background needed to understand the state-machine encoding, and other relevant topics.

2.1 Dependent type theory

Given functions between sets $A, B : \mathbf{Type}$, $A \rightarrow B$, we can generalize this by changing $B : \mathbf{Type}$ to be a function $B : A \rightarrow \mathbf{Type}$, then letting us write the dependent function $(x : A) \rightarrow B(x)$. We can also do the dual construction, and get dependent pairs. The inference rules for these can be seen in Figure 1. These rules should be familiar as they are the analogous to the rules for $\forall, \exists$ from the natural deduction system, their relationship can be seen in Figure 2. If you are unfamiliar with dependent types, it is often helpful to view them through this relationship. For a more in depth look at dependent type theory, read [7].

$$\begin{array}{c}
 \frac{A : \mathbf{Type} \quad B : A \rightarrow \mathbf{Type}}{(x : A) \rightarrow B(x)} \Pi f \quad \frac{\Gamma, x : A \vdash v : B(x)}{\Gamma \vdash \lambda x. v : (x : A) \rightarrow B(x)} \Pi i \quad \frac{\Gamma \vdash f : (x : A) \rightarrow B(x) \quad \Gamma \vdash v : A}{\Gamma \vdash f v : B[x / v]} \Pi e \\
 \text{(a) } \Pi\text{-forming rule} \quad \text{(b) } \Pi\text{-intro rule} \quad \text{(c) } \Pi\text{-elimination rule} \\
 \\
 \frac{A : \mathbf{Type} \quad B : A \rightarrow \mathbf{Type}}{(x : A) \times B(x)} \Sigma f \quad \frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B[x / a]}{\Gamma \vdash \langle a, b \rangle : (x : A) \times B(x)} \Sigma i \quad \frac{\Gamma \vdash v : (a : A) \times B(a) \quad \Gamma \vdash a : A, b : B[x / b] \vdash x : C}{\Gamma \vdash \text{let } \langle a, b \rangle := v \text{ in } x : C} \Sigma e \\
 \text{(d) } \Sigma\text{-forming rule} \quad \text{(e) } \Sigma\text{-intro rule} \quad \text{(f) } \Sigma\text{-elimination rule}^3
 \end{array}$$

Figure 1: Inference rules for Π and Σ types

$$\Pi, \Sigma \xrightleftharpoons[\text{Categorification}^4]{\text{propositional truncation}} \forall, \exists$$

Figure 2: Curry-Howard correspondance

2.2 Lean

Lean [4] is a popular dependently typed proof assistant. Lean is famous for multiple reasons, 3 of them are:

1. It has Mathlib [2], one of the world's largest repositories of formalized mathematics.
2. It has a rich tactic language in which most proofs are written.
3. It has a compiler, and can be used as a general purpose programming language.

As a consequence of 3. we have a tension between the logical and computation sides of a proofs/programs. A classic example of where this occurs is in compiling the elimination principle for strong-induction⁵. We talk about this being a difference between the kernel (the small safe piece of code we believe to be correct) and the compiler (which is much larger and more complex).

Lean requires all induction to be structural, so to do induction over a general well-founded relation, we need some inductive datatype (Section 2.4), Lean calls this `Acc` and defines it as seen in Figure 3a. On this Lean defines a function `WellFounded.fixF` (Figure 3b), which allows for strong induction. This

³Technically this is not the most general eliminator for Sigma's, as C can depend on the content of the type. For symmetry and simplicity I omit this.

⁴S. Awodey [8] calls this 'categorification', as it is only one direction of Curry-Howard. Figure 2 can also be said as 'Curry-Howard', but I prefer to highlight the directions of the relationship.

⁵This is now, for the case of natural numbers, fixed as of lean 4.27. <https://www.youtube.com/watch?v=LOUbbiV0mWc> <https://lean-lang.org/doc/reference/latest/releases/v4.27.0/>

function is quadratic in its runtime, therefore as a consequence the kernel takes quadratic runtime to execute.

```
inductive Acc {α : Sort u} (r : α → α → Prop) : α → Prop where
/--
A value is accessible if for all `y` such that `r y x`, `y` is also accessible.
Note that if there exists no `y` such that `r y x`, then `x` is accessible.
Such an `x` is called a _base case_.
-/
| intro (x : α) (h : (y : α) → r y x → Acc r y) : Acc r x
```

(a) Definition for Acc taken from Lean’s GitHub [4]

```
def WellFounded.fixF
{α : Sort u} {r : α → α → Prop} {C : α → Sort v}
(F : (x : α) → ((y : α) → r y x → C y) → C x) (x : α) (a : Acc r x)
: C x := ...
```

(b) Type signature of WellFounded.fixF

Figure 3: Some strong-induction primitives from Lean

It would be unacceptable for a program using strong recursion to be quadratic for a general purpose programming language, therefore the compiler removes the usage of fixF, and generates the expected calls in the intermediate representation (LCNF).

We can see that given the source-code Figure 4a, it generates a definition Figure 4b with Nat.fix (a variant of fixF), but in the output LCNF we get Figure 4c, which does not contain any mention of Nat.fix, but we see the recursive call, demonstrating how the logical and computation definitions diverge.

```
set_option trace.Compiler.result true in
@[semireducible]
def gcd (a b : Nat) : Nat :=
  if h' : b = 0 then a
  else
    have : a % b < b := Nat.mod_lt a
      (Nat.pos_of_ne_zero h')
    gcd b (a % b)
termination_by b
```

(a) GCD source code

```
/--
info: def gcd._unary : (Nat) ×' Nat →
Nat :=
WellFounded.Nat.fix (fun x =>
PSigma.casesOn x fun a b => b) fun _x a =>
  PSigma.casesOn (motive := fun _x =>
    ((y : (Nat) ×' Nat) → InvImage
      (fun x1 x2 => x1 < x2) (fun x =>
        PSigma.casesOn x fun a b => b) y _x → Nat)
    →
      Nat)
    _x
    (fun a b a_1 =>
      if h' : b = 0 then a
      else
        have this := ...;
        a_1 (b, a % b) ...)
    a
  -/
#guard_msgs in
#print gcd._unary
```

(b) generated kernel code

```
[Compiler.result] size: 9
def gcd a b : tobj :=
  let _x.1 := 0;
  let _x.2 := Nat.decEq b _x.1;
  cases _x.2 : tobj
  | Bool.false =>
    let _x.3 := Nat.mod a b;
    dec a;
    let _x.4 := gcd b _x.3;
    return _x.4
  | Bool.true =>
    dec b;
    return a
```

(c) Generated intermediate representation

Figure 4: Gcd as Source, Kernel and LCNF

This general trick inspired the project, as it demonstrates that there is precedent in Lean4 to have separate logical and executorial models. This is further highlighted by the existence of the `@[implemented_by]` attribute, which lets the programmer off-load the runtime behaviour of a function to another possibly unsafe definition. When a definition is marked as `@[implemented_by x]`, the code-generator treats it as `x` when compiling the code. Safety of programs using `@[implemented_by]` is a complex endeavour to verify, as Lean has a tactic called `native_decide` that runs the generated code outside of the kernel.

When talking about lean we also use the term *definitionally equal* to mean β, η -equivalent, and *propositionally equal* if two terms are identified by a generalized algebraic data-type. Additionally we have *heterogeneous equality* where the types are only propositionally equal.

2.3 Polynomial functors

Definition 2.3.1: A functor is pair of a map $(\alpha : \text{Type}) \rightarrow F \alpha$ sending types to types, and a map between types to functions between the functor at those types $(\alpha \beta : \text{Type}) \rightarrow (\alpha \rightarrow \beta) \rightarrow F \alpha \rightarrow F \beta$, such that the map distributes over composition and preserves identities.

Example: An example of a functor in programming is `Lists`, given a `List String` we can use the function `len : String → Nat`, to make a function `map len : List String → List Nat`:

Polynomial functors are functors with a collection of constructors (the head type), where the children correspond to how many of the polymorphic argument are wanted. Alternatively they are functors generated from taking Sums and Products over constants and projections. The formal definition can be seen in Figure 5.

Definition 2.3.2: A univariate polynomial functor $P(\alpha)$ is given by: a head-type H , along with a child family c_h parameterised by the head $h : H$. A polynomial functor applied at type α , is a sigma-type of the head, and the child family [9]. As seen in Section 2.1, the object of this type will be a pair of a head and child functions.

$$P(\alpha) \triangleq (h : H) \times c_h \rightarrow \alpha$$

Figure 5: Formal definition of polynomial functor

This dissertation will mainly focus on multivariate polynomial functors. Instead of having a single child, multivariate polynomial functors have a n -tuple of children for an n -polynomial functor.

Polynomial functors have all finite products (Figure 7, Figure 8) and coproducts (Figure 7, Figure 9). Polynomial functors have fixed points corresponding to **inductives**, and cofixed-points as **coinductives** [5], [6]. In the figures there are ellipsis, in reality multivariate polynomials operate on Type-vectors which come from the category formed by the products $\mathbf{Type}^n$ for some $n : \mathbb{N}$, these are not ergonomic to work.

2.3.1 Common polynomials

Some examples of common polynomials are projection Figure 6, constant functors Figure 7, binary products Figure 8, and binary sums Figure 9. One crucial polynomial not shown here is composition. The code of composition is not very insightful. Composition takes an n -polynomial, and n lots of m -polynomials, and outputs an m -polynomial. For example, taking the 3-polynomial F and 3 2-polynomials G_1, G_2, G_3 , the constructor of the composition takes a $F(G_1 \alpha \beta)(G_2 \alpha \beta)(G_3 \alpha \beta)$. The polynomial notation for this can be found in [2], [6]. For those familiar, the composition polynomial works exactly like composition in primitive recursive functions.

```
inductive Prj (i : Fin n) (a_1 ... a_n : Type)
  where
  | mk (t : a[i])
```

(a) Inductive Notation

$$\lambda(i : \underline{n}). \langle 1, \lambda j \ h. \text{ if } i = j \text{ then } 1 \text{ else } 0 \rangle$$

(b) Polynomial

Figure 6: Projection polynomial

```
inductive Const (T : Type) (a_1 ... a_n : Type)
  where
  | mk (t : T)
```

(a) Inductive Notation

$$\lambda(T : \mathbf{Type}). \langle T, \lambda i \ h. 0 \rangle$$

(b) Polynomial

Figure 7: Constant polynomial

```
inductive Prod (A B : Type) where
| mk : A → B → Sum A B
```

(a) Inductive Notation

$\langle 1, \lambda i \ () . 1 \rangle$

(b) Polynomial

Figure 8: Product polynomial

```
inductive Sum (A B : Type) where
| inl : A → Sum A B
| inr : B → Sum A B
```

(a) Inductive Notation

$\langle \mathbb{B}, \lambda \begin{pmatrix} 0 & \text{true} & \mapsto 1 \\ 1 & \text{false} & \mapsto 1 \\ - & - & \mapsto 0 \end{pmatrix} \rangle$

(b) Polynomial

Figure 9: Sum polynomial

2.4 Recursion and Inductive datatype

Definition 2.4.1: An inductive datatype, is a type with a collection of constructors, and an eliminator consuming the tree and produces some data. Objects of inductive datatypes are **well-founded** trees of data.

Recalling algebraic datatypes from languages such as OCaml, Haskell and Rust. These are structures freely generated from a set of constructors. The classic example is a list, given as two constructors, `val cons : 'a → 'a list → 'a list` and `val nil : 'a list`, and a way to consume lists `val fold : 'a → ('b → 'a → 'a) → 'b list → 'a`. We write this as seen in Listing 1. If we restrict these datatypes to well-founded trees, these are exactly inductives⁶. Any pure function that terminates, must return a finite tree of constructors. Fold is guaranteed to terminate for any finite tree

In Lean, recursive functions are compiled into a call to some recursor for a type. For lists this recursor is a dependent generalization of `fold`. All structurally recursive functions can be compiled into a call to `fold`. We also have two operations `mk : F (μ F) → F` and `dest : μ F → F (μ F)`. These form an equivalence (J. Lambek [10] Corollary 2.4), giving us the result that the cofix-point of `F`, written as `μ F`, is equivalent to `F (μ F)`. In practice this means we can unfold `μ F`.

```
type ('a, 'b) listF = Nil | Cons of 'a * 'b
type 'a ilist = Mk of (('a, 'a ilist) listF)

let mk x = Mk (x)
let dest = function | Mk v → v
let rec fold bh ih = function
| Mk(Nil) → bh | Mk(Cons(h, t)) → ih h (fold bh ih t)
```

Listing 1: Definition of list in OCaml

2.5 Corecursion and coinductive types

Definition 2.5.1: A coinductive datatype is a type with a collection of constructors, and a generating function. Objects of a coinductive datatypes are potentially infinite trees of data.

Where inductive datatypes have a `fold : (('a, 'b) f → 'b) → (('a, 'g) f as 'g) → 'b` (rec) to consume them, coinductive types have an `unfold : ('b → ('a, 'b) f) → 'b → (('a, 'g) f as 'g)` (corec) to produce them. Intuitively, view it as if a fix `F` yields a bounded structure, then cofix `F` yields an unbounded structure⁷. Another way to view coinductive types are as a state-machine, here the corecursor takes center stage. We can say a coinductive datatype is a datatype generated by some corecursor state-machine.

⁶In OCaml, Haskell and Rust, datatypes do not correspond directly to inductive types since we can have non-well-founded trees, an example is `let rec degen = Cons((), degen)`.

⁷That is `fix (1 + (A × −))` gives lists, then `cofix (1 + (A × −))` gives potentially infinite lists.

2.5.1 In OCaml

We can make lazy lists by thunking recursive occurrences, this thunking allows infinite structures to exist in finite memory. Then all we need to do is add a definition of `unfold`, `mk` and `dest`. All these can be seen in Listing 2. We also have constructors and destructors forming inverses as for recursive datatypes⁸.

```
type 'a icolist = CMk of (unit → ('a, 'a icolist) listF)

let mk x = CMk (fun _ → x)
let dest = function | CMk v → v ()
let rec unfold gen v = CMk (fun _ →
  match gen v with
  | Nil → Nil
  | Cons(h, t) → Cons(h, unfold gen t))
```

Listing 2: Definition of colist in OCaml

The keen eyed will notice and call out that `unfold` is not structurally recursive, nor even decreasing at all. This is the key difference between inductive types and coinductive types; coinductive types can be ‘infinitely big’. Despite not being terminating, the definition is *productive*. Termination is having a complete structure in finite time, productivity is being able to always compute the next layer in finite time.

Definition 2.5.1.1: Productivity is the property of a corecursive function to always be able to compute the next layer in finite time.

2.5.2 Streams

Streams are similar to lists. The difference is they have no ‘nil’ constructor. An inductive datatype of this form is empty⁹, though it is inhabited for coinductive datatypes. Streams and colists are also common in general purpose programming, Java has `Stream`, Rust and Python have `Iterator`. This backs up our use cases put forward in the introduction. We will give concrete implementations of them for each of the encodings as follows.

2.5.3 The M -type

The cofixed-point of a polynomial functor, is called an M -type¹⁰ possibly infinitely deep trees in which each layer is an unfolding of some polynomial F . This means the implementations must have both a corecursor `corec` : $\{\alpha : \text{Type}\} \rightarrow (f : \alpha \rightarrow F \alpha) \rightarrow \alpha \rightarrow M F$, and a destructor `dest` : $M F \rightarrow F (M F)$. These are the functions mentioned in Section 2.5.1. Often we also want to ship a way to `mk` a new layer, but we can define this from the two operators above.

There are different approaches to encode M -types in Lean. We will focus on two encodings of M -type’s. The state-machine encoding (Section 2.5.4) and Progressive Approximation (Section 2.5.3.1) (as seen in Mathlib [2]).

2.5.3.1 Progressive approximation encoding

A simple way to encode M -types, are as functions emitting finite and growing trees that must at every level agree. This corresponds to the limit over the natural number category as the dual of J.

⁸The dual of Lambek’s theorem [10]

⁹See the following lean

```
inductive SS (α : Type) where | cons : α → SS α → SS α
def SS.toEmpty : SS α → Empty | .cons _ tl => toEmpty tl
```

¹⁰Often phrased as types who’s semantics are terminal F -coalgebra.

Adámek [11] (see section “Free algebra algorithm”). Agreement is given by them being the same up to the previous depth as seen in Figure 10. Agreement can be thought of as the structure ‘crystalising’, since it exclusively ‘grow’ from the crystal edge.

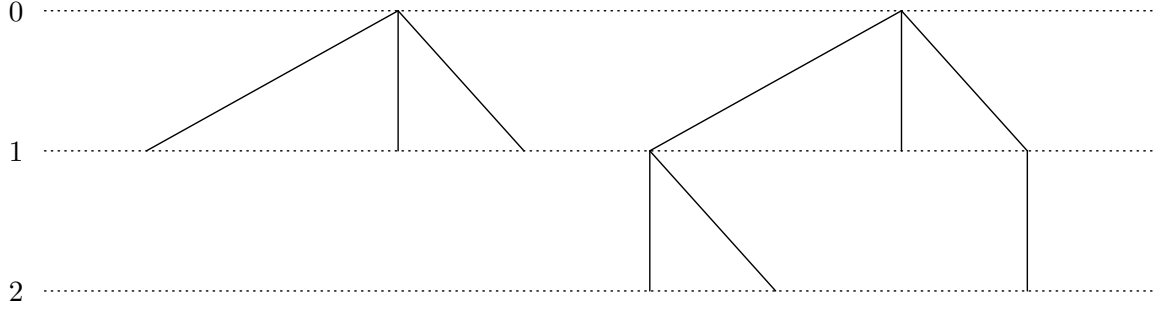


Figure 10: Agreement of two trees of height 1 and 2

In Lean, for a specific functor, we begin setting up the definition of approximation. With this we define the coinductive type as the agreeing functions.

```
def StreamF (α ρ : Type _) := α × ρ

inductive StreamF.Approx {α : Type} {base : Type} : {n : Nat} → {m : Nat}
  → n.repeat (StreamF α) base → m.repeat (StreamF α) base → Prop where
| base : @Approx _ _ 0 (m + 1) b v
| consS : @Approx _ _ n m t1 t2 →
  @Approx _ _ (n + 1) (m + 1) ⟨h1, t1⟩ ⟨h1, t2⟩

structure PA (α : Type _) : Type 0 where
  obj : (n : Nat) → n.repeat (StreamF α) Unit
  cryst : ∀ n, StreamF.Approx (obj n) (obj (n + 1))
```

Listing 3: Definition of approx for stream

For the corecursor, we need to implement a function to emit the finite and growing trees, and a proof that the trees agree. This is also where the performance hit is incurred, as we have to regenerate the tree every time we want to reach the new depth.

```
def corec.o (gen : ρ → StreamF α ρ) (v : ρ) : (n : Nat) → n.repeat (StreamF α) Unit
| 0 ⇒ .unit | n + 1 ⇒ (gen v).map id (o gen . n)
def corec (gen : ρ → StreamF α ρ) (init : ρ) : PA α where
  obj := corec.o gen init
  cryst n := by
    induction n generalizing init
    · exact .base
    case succ n ih ⇒
      unfold corec.o
      rcases gen init with ⟨a, b⟩
      change StreamF.Approx (n := n+1) (m := n+2) ⟨a, corec.o gen b n⟩ ⟨a, corec.o gen b (n + 1)⟩
      exact StreamF.Approx.consS (h, := a) (ih b)
```

Listing 4: Objects of the corecursor for streams

The destructor builds an instance of the functor, where the correct path is selected from the corecursor.

```
def dest (x : PA α) : StreamF α (PA α) := ⟨
  (x.obj 1).1,
  (x.obj .succ ▷ .snd),
  fun n ⇒ by
    have := x.cryst (n+1)
    dsimp
    generalize (x.obj (n + 1)) = a, (x.obj (n + 1 + 1)) = b at this;
    rcases a; rcases b; rcases this with (|ih)
    exact ih
⟩
```

Listing 5: Destructor for streams

Proving the coinduction principle (Section 2.6) is harder. We do not need to worry about implementing and proving this as Mathlib [2], [5] already provides an implementation of this.

2.5.4 State-Machine encoding

Given some polynomial functor P , the state-machine encoding of $\mathbb{M} P$ is given by: some carrier type $\alpha : \text{Type}$, a function (coalgebra) $f : \alpha \rightarrow P \alpha$, and some initial state $a : \alpha$. Once this is done there are 2 key problems:

1. The object constructed is only weakly terminal and equivalently no coinduction principle,
2. the object resides in a higher universe.

Focusing on the first 1st problem, this is solved by quotienting over bisimilarity. The reason this works has to do with how bisimilarity ‘hides’ the generating type. This makes destructuring the only operation you can do to access the type. The 2nd problem is tackled in Section 3.4.

2.6 Bisimilarity

Definition 2.6.1: Bisimilarity is an equivalence relation on coinductive types. Two coinductive objects are bisimilar if, for every possible choice in a structure A, there exists a corresponding choice in structure B, and vice versa, such that you can repeat this procedure.

Bisimilarity is not a well-founded relation. This means we need to define it as a coinductive predicate instead of an inductive one. Then the proof principal for this will be giving an invariant, showing the invariant is a bisimulation, and the initial values are contained within the invariant.

A classic example would be to check if the two state-machines are equal. In Figure 11 you see two state-machines for two different regexs. One could use bisimilarity to show these are the same. The invariant for this would probably be $(a = q_0 \wedge b = q_0) \vee (a = q_0 \wedge b = q_2) \vee (a = q_1 \wedge b = q_1) \vee (a = q_1 \wedge b = q_3)$. This also contains the initial state $a = q_0 \wedge b = q_0$. We also need to show the invariant is a bisimilarity which is a bit more involved.

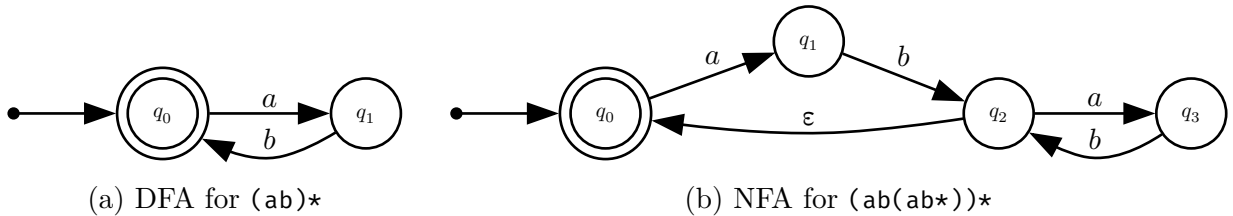


Figure 11: Two bisimilar state-machines

On streams, a bisimulation equates heads, and relates tails. This can be seen in Listing 6. Coinductive types have coinduction principle `bisim : Bisim a b → a = b`. In Rocq this has to be an axiom, in lean it is provable. M. P. Fiore [12] is an insightful read on this topic.

```

Quotient.mk _ (.corec gen init)

@[simp]
theorem corec.hd {A Carr : Type _} {gen : Carr → A × Carr} {init : Carr}
  : (corec gen init).hd = (gen init).fst := rfl

```

Listing 6: Bisimilarity on streams

2.7 The Free Monad

As mentioned in Section 2.5, every corecursive function has to be productive. In the current library definition, this means we need to define the function using the corecursor `corec {β α} : (β → P (α :: β)) → β →  $\mathbb{M} P \alpha$` . This can be cumbersome as one can only define one layer of the coinductive datatype at a time. One can image that after the first layer (which is required for productivity), we have a

choice to either `recall` the corecursor from that point, or continue a deeper structure (visualized in Figure 12). A structure like this would allow for defining multiple layers of a coinductive.

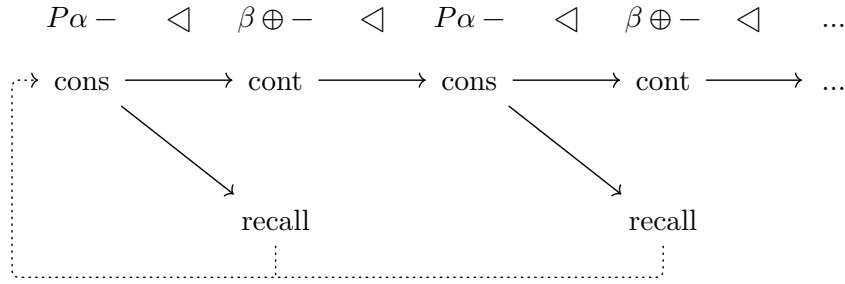


Figure 12: Choice points for a producing a stream

Making a structure like this is possible with a bit of setup.

Definition 2.7.1: A Monad is a functor M , with an additional operator $>>= : (\alpha \rightarrow M \beta) \rightarrow M \alpha \rightarrow M \beta$, and a function `return` : $\alpha \rightarrow M \alpha$ satisfying the rules `return >>= a = a`, `f >>= return v = f v` and `x >>= f >>= g = x >>= λx ↦ f x >>= g`.

Given a monad M , we can simply interpret it as a functor, but what if we want to go the other way; take a F and make the most general (or *universal*) monad out of it. The process for this involves keeping ‘as much structure as possible’, one way of doing this is have two constructor `return` : $\alpha \rightarrow \text{Free } F \alpha$, and `bind` : $F (\text{Free } F \alpha) \rightarrow \text{Free } F \alpha$. When doing this construction as a coinductive it is called the *Free-monad*. This has a bind operator that corresponds to pasting trees together. When referring to it, it will be generalized to a $n + 1$ -polynomial, and written $\text{Free } P \alpha \beta$, the definition of this will be given in the implementation section.

Using the Free monad one can define a futumorphism `futu {β} : (β → P (α :: Free P α β)) → M P α` [13], [14], [15]. This turns out to be exactly the structure wanted.

2.8 Interaction trees

Denotational semantics stands as an alternative to operational semantics, where rather than having some relation of program steps, one maps syntax to some mathematical object which can reason about directly. For lambda calculi we have categorical semantics, but for imperative languages it can be much harder to find a denotational object.

Interaction trees [1] are a form of free-monad, in which you have a set of visible effects. *ITrees* have 3 constructors, returning a value (monad unit), having a silent transition (for non-termination), and emitting a visible effect. The *Rocq* definition can be seen in Listing 7. The effects of the *ITree* correspond to the effects of the imperative language. Some examples could be printing, reading and writing to variables, and reading user input. *ITrees* have a ‘monadic interpretation’ where the effects are transformed to operations in some monad. Further, *ITrees* have an equivalence relation called *Weak Bisimilarity*, this is bisimilarity modulo silent steps, two objects are related if they have the same visible effects and return values.

We will speak more about *ITrees* in Section 3.6

```

CoInductive itree (E: Type -> Type) (R: Type): Type :=
| Ret (r: R)
| Tau (t: itree E R)
| Vis {A: Type} (e : E A) (k : A -> itree E R)
  
```

Listing 7: *Rocq* defintion from [1]

2.9 Methodology

Writing in a proof assistant is different to writing in a general purpose language, this is especially the case in Lean as it is a hybrid of both styles. A notable and immediate difference is the prevalence of software testing of programs. Instead of having tests that can be flakey and sparse, in Lean you have checked proofs.

In a conventional programming language, you can follow Test Driven Development where you constantly refine the specification with additional tests, which you check as you go through the code. In Proof Driven Development (PDD, [16]) you work similarly to TDD, but now when you have your function, instead of knowing it works for finite cases, you can be guaranteed that it satisfies the equation. Furthermore, the proofs can serve to enrich the main artifact. This will be seen in the ITree implementation as well as the futumorphic productivity. Lean makes working in this style a delightful experience using constructs such as `#check`, `#print` and `#eval`, one can see exactly what a function is and what it is doing. These can also be turned into conventional tests using `#guard_msgs in`, which raises a compile error upon an output changing. PDD was originally demonstrated for Idris, which in many ways is similar to Lean as it is a hybrid language, therefore I will apply this approach as I write the code.

Lean also has great automation features which I use at many points in the dissertation, these are the unification solver `exact?`, SMT solver `grind`, and the simplifier `simp`. These all stand as principled ways to develop programs and proofs that would otherwise be tedious for a human. I use these regularly to speed up the development process. When working with `simp` one has take care for non-terminal `sims`, as in `sims` that do not close a goal, for these we use the `simp?` tactic to limit the set of available lemmas. The reason for this has to do with how adding more lemmas can break the normal form `simp` leads to. Lean's built in linter can handle this.

2.10 Tools used

To aid the development and communication process I used `git`, hosting the repository on GitHub. I also shipped a compiled version of this dissertation publicly, so my supervisors could read it and have it update live on the web. The dissertation is written in `typst` as it is a feature rich, well documented, and popular TeX alternative.

2.11 Requirement Analysis

To expand on the success criteria given in the proposal, I specified the project using MoSCoW. This will be used to reference what sections implement what features of the project. Here the success criteria have become MUSTs, and improvements have become SHOULD and COULD. I have done this for the core and each of the extensions of the project.

2.11.1 State-machine encoding (Core)

- SM1 The SME Stream MUST be implemented in its high universe form.
- SM2 The equivalence between SME Stream and PA Stream MUST be implemented.
- SM3 The SME M MUST be implemented in its high universe form.
- SM4 The SME M MUST be constant time under destructuring.
- SM5 The equivalence between SME M and PA MUST be implemented.
- SM6 The SME M MUST be able to express the NT monad, Section 2.11.2.
- SS1 The SME M SHOULD have a coinduction principle.

- SC1 The SME M COULD have a low universe shifted version Section 2.11.3.
- SC2 The SME M COULD have an Interaction tree library Section 2.11.4.
- SC3 The SME M COULD have a Productivity Transform Section 2.11.5.
- SC4 The SME M COULD be zero cost.
- SW1 The SME M WONT have a syntax macro as in [5].
- SW2 There WONT be any work towards coinduction-up-to systems.

2.11.2 Non-termination-monad (Core)

- NM1 The NTMonad MUST be implemented using the SME.
- NM2 The NTMonad MUST have a monadic bind and return.
- NS1 The NTMonad SHOULD be a lawful monad.
- NC1 The NTMonad COULD be set up to use Section 2.11.5.

2.11.3 AltRepr Type (Extension)

- AM1 The AltRepr Type MUST be implemented.
- AM2 The AltRepr Type MUST have an eliminator.
- AM3 The AltRepr Type MUST have the computational behaviour of the higher type.
- AC1 The AltRepr Type COULD be zero-cost.

2.11.4 Interaction Trees (Extension)

- IM1 ITrees MUST be implemented.
- IM2 ITrees MUST have strong bisimilarity.
- IM3 ITrees MUST have bind and return constructs.
- IM4 ITrees MUST have KTrees (continuation trees).
- IS1 ITrees SHOULD have flow combinators implemented.
- IS2 ITrees SHOULD have a coinduction principle.
- IS3 ITrees SHOULD be a lawful monad.
- IC1 ITrees COULD have weak bisimilarity implemented.
- IC2 ITrees COULD have the monadic interpretation.
- IW1 ITrees WONT implement the family of effects put forward in [1].

2.11.5 Free Monad (Extension)

- FM1 Free Monad MUST have an in-universe futumorphism.
- FM2 Free Monad MUST have an injector.
- FS1 Free Monad SHOULD have proof-principles for unfolding the futumorphism.
- FS2 Free Monad SHOULD have proof-principles for the injector.
- FC1 Free Monad COULD have a cross universe futumorphism.
- FW1 Free Monad WONT be hetrogenous [17].

Chapter 3: Implementation

This chapter describes the implementation of each of the requirements stated in Section 2.11. I will break it down into a common section, then go by individual components. I will also mention which parts are in process of being upstreamed and which parts are already in Mathlib [2].

The Lean component of this repository, and how they relate to each of the requirements can be seen in Figure 13. Additionally the import graph can be seen in Section 6.A.

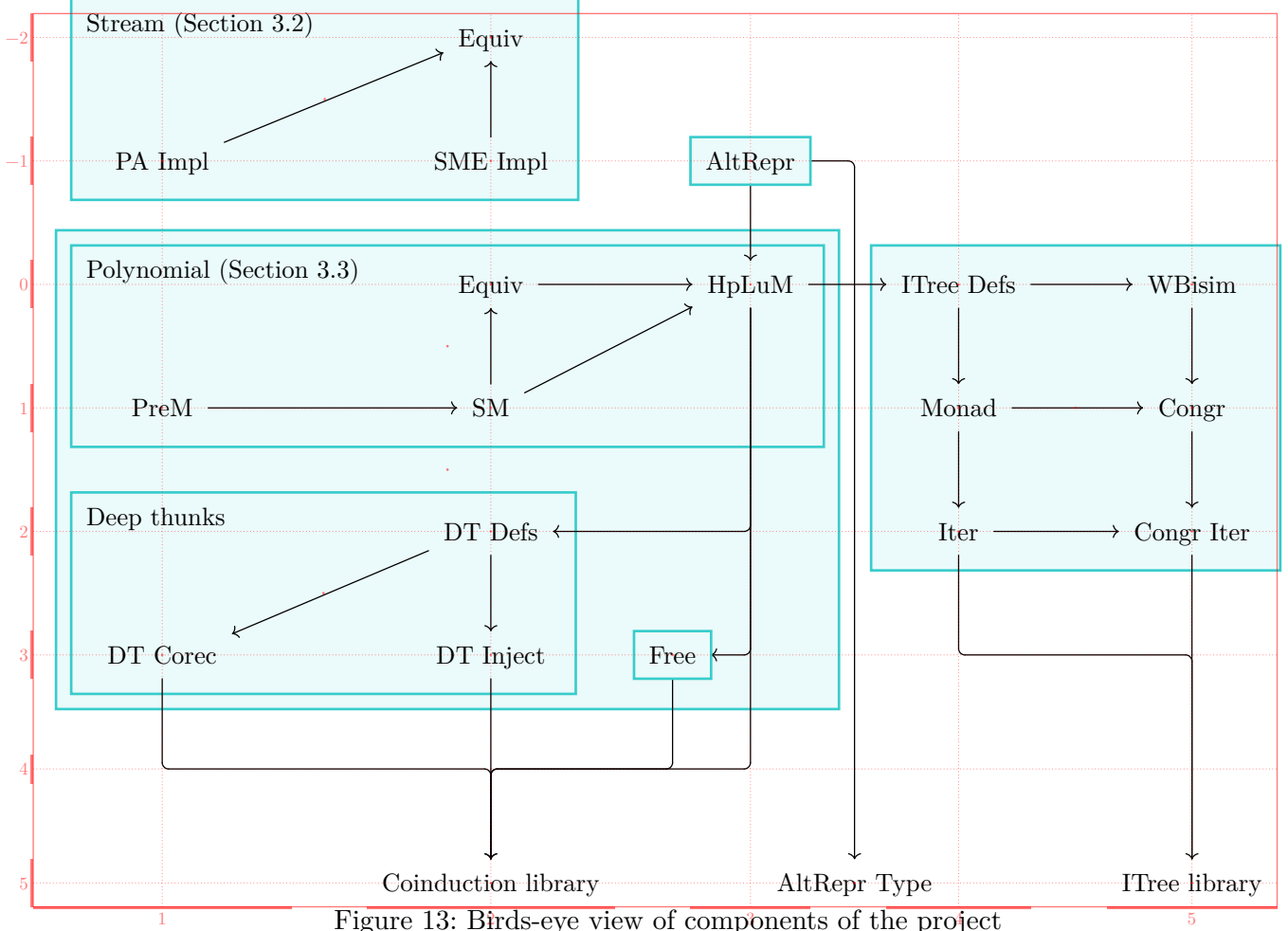


Figure 13: Birds-eye view of components of the project

The main component implemented in this dissertation is State-machine M -type **SM**, along with the equivalence $\text{SM}.\{\mathcal{U}, \max \mathcal{U} \mathcal{V}\} \text{ P } \alpha \approx \text{M}.\{\mathcal{U}\} \text{ P } \alpha$. This will be used to implement the **HPuM**, which is the best of both worlds.

3.1 Common

Many large components of this project do not fit into any of main components. This section is dedicated to this exact kind of component.

3.1.1 Transliteration

Definition 3.1.1.1: A transliteration over some parameter $\text{span}^{11} X$, either in some **Type** or more commonly over some universe $\mathcal{U}$, is a family of functions $t_{a,b} : Xa \rightarrow Xb$. Such that

1. The functions are closed under composition $t_{b,c} \circ t_{a,b} = t_{a,c}^{12}$,
2. and the self-path is identifiaion $t_{a,a} = \text{id}$.

A family of functions, that keep solving problems throughout this dissertation are what I call transliterations.

The trivial instantiation of a transliteration is a `cast`¹³. Here we pick X as equal types. Obviously `cast aa = id` and `cast bc ∘ cast ab = cast (ab.trans bc)`. One could even say that a transliteration is a function that behaves like `cast`.

An example we will keep defining is universe transliteration, here we take $X = \text{ULift}_{(-)}$, using this we define the transliteration as seen in Listing 8. This is the closest we get to having omega-types and cumilativity in Lean; as long as the function is applied at a usage of `ULift` it all works. We will use this function to define transliteration between universe-lifted polynomials, we will see more of this in Section 3.1.2.1.

```
universe u v w
variable {A : Type u} {x : ULift A} {a : A}

@[inline] def transliterate : ULift.{v} A → ULift.{w} A := ULift.up ∘ ULift.down

@[simp] theorem transliterate_idempotent
  : transliterate ∘ transliterate = transliterate (A := A) := rfl
@[simp] theorem transliterate_noop
  : transliterate x = x := rfl
```

Listing 8: Transliteration between universes of types

Another major usage of a transliteration was used when defining the eliminator for `AltRepr`, I will discuss this more when we get to Section 3.4. This was where I first found transliterations, and made it possible to define a universe-polymorphic eliminator.

3.1.2 Expanding the progressive approximation theory

`sme/Sme/ForMathlib/{PFuncutor/*,TypeVec.lean}`

During the feasibility assesment I noticed that, in the current formalised theory of polynomials, the equivalence would not even type-check. This stemmed from a problem with the corecursive principle for the M -type in the old implementation. `corec : {α : TypeVec.{ℳ} n} → {β : Type ℳ} → (g : β → P (α :: β)) → β → M P α14 Mathlib [2]. The problem here is that both  $\alpha$  and  $\beta$  have to both reside in  $\mathcal{U}$ . Solving this is done through the next two sections.`

3.1.2.1 Universe lifting of polynomial functors.

The main problem caused here comes from the fact that Lean is not cummulative. This means it is impossible to express a universe heterogeneous typevector. In other words $\alpha :: \beta$ is only typable if $\alpha : \text{TypeVec}.\{\mathcal{U}\}n$ and $\beta : \text{Type } \mathcal{U}$. The natural way of solving this is using the supremum in universe levels you get from `ULift : Type ℳ → Type (max ℳ ℳ)`. This means we can have $\beta : \text{Type } \mathcal{U}$ and $\alpha :$

¹¹This is not necessarily a type, as Lean does not have omega-types (`Setω` from Agda) [18].

¹²This is similar to saying it is an idempotent, but technically $t_{b,c}$ and $t_{a,b}$ are not the same function.

¹³Or if you read the HoTTbook [7], this is called `transport`.

¹⁴<https://github.com/leanprover-community/mathlib4/blob/7a60b315c7441b56020c4948c4be7b54c222247b/Mathlib/Data/PFuncutor/Multivariate/M.lean#L152-L154>

Type $\mathcal{V}$, then ulift both of them to a common universe $\text{ULift } \alpha :: \text{ULift } \beta : \text{TypeVec}.\{\max \mathcal{UV}\}(n + 1)^{15}$.

The next hurdle we encounter is that polynomials are restricted to a universe level. Recall the definition from Section 2.3. Observe how for a $\text{MvPfunctor}.\{\mathcal{U}\}n$, we require that both the head and child reside in $\mathcal{U}$. This will also cause problems, as looking back at the definition of the corecursor, we will require P to be able to accept $\text{ULift } \alpha :: \text{ULift } \beta$. If we do not add the ability to lift P , the unifier will force $\mathcal{U} = \mathcal{V}$, thereby invalidating all the work we did in the previous section. We define it as $\text{ULift } P \triangleq \langle \text{ULift } P.1, \lambda x \mapsto \text{ULift } (P.2x.\text{down}) \rangle$. This works and now we can move on to generalizing the corecursor. This definition also satisfies naturality.

3.1.2.2 Generalizing the corecursor

`[UPSTREAMED], sme/Sme/PFunctor/ForMathlib/M.lean`

Now with all the work in the previous section, first we generalize a helper function¹⁶, then we can define $\text{corecU} : \{\alpha : \text{TypeVec}.\{\mathcal{U}\}n\} \rightarrow \{\beta : \text{Type } \mathcal{V}\} \rightarrow (g : \beta \rightarrow \text{ULift } P (\text{ULift } \alpha :: \text{ULift } \beta)) \rightarrow \beta \rightarrow \text{M}.\{\mathcal{U}\} P \alpha$. Notably we are able to fit the object into $\mathcal{U}$ (this will not be the case for the SME).

The functions corecU and dest satisfy an unfolding equation. This is more complex than it used to be as we now need to lower the universe before we continue with the mapping.

3.1.3 Polynomial equivalents

`sme/Sme/PFunctor/EquivP.lean`

Often when people talk about “polynomials”, they are referring to structures that are equivalent some polynomial. As in a `List` is not a pair of a head and child types, but it is equivalent to a polynomial of this form. This is for example how A. Keizer [5] refers to polynomials. I found it useful to work like this as well, for this I made a type-class `EquivP` heavily inspired by how it was done in A. Keizer [5] and `Mathlib` [2]. I took the definition of curried type-functions (CTFs) from `QPFTypes` [5] and implemented coherences for these for the common polynomials. I used this to make it cleaner to work with `ITrees` when I reached that point. A key difference is that I implemented it using the output CTF as an `outParam`, this means that I can synthesize the CTF from a given polynomial allowing for an easier user interface. Type class synthesis works similar to a prolog engine and `outParam` corresponds to the prolog mode ?.

3.1.4 Natural isomorphisms of MvFunctors

`sme/Sme/PFunctor/NatIso.lean`

A problem I found when working with isomorphisms of $\text{Mv}(P)\text{Functors}$ was that isomorphisms need numerous coherences proven about them. I observed that the coherences I was proving corresponded to naturality; it satisfied $f \langle \$\$ \rangle \text{iso } x = \text{iso } (f \langle \$\$ \rangle x)$. To solve this I defined natural isomorphisms of multivariate functors Listing 9. The naturality of the inverse of the isomorphism is provable.

```
structure NatIso {n} (P Q : outParam (TypeVec.{v} n → Type u))
  [MvFunctor P] [MvFunctor Q] where
  equiv : ∀ {α}, P α = Q α
  nat' {α β} {f : α → β} (x : P α) : f <$$> equiv x = equiv (f <$$> x)
```

Listing 9: Natural isomorphisms on MvFunctors

3.2 Stream implementation

`sme/Sme/Stream/*`

SM1, SM2

¹⁵Note we overload `ULift` as a notation to refer to lifting other structures as well.

¹⁶Done in PR #30400 to `Mathlib` [2]

As proving these equivalences is challenging I decided I would start by implementing it for the special case of streams. Streams are the text-book coinductive datatype that most people know as mentioned in Section 2.5. Therefore I expected this to be pedagogical to implement.

3.2.1 State-machine streams

sme/Sme/Stream/SDefs.lean

SM1

First I set up a class of prestreams, these correspond to the corecursive principle holding their type. I had to define `hd` and `tl` for streams here corresponding to the destructors of streams. Care had to be taken to ensure that the definition would work under a quotient as well.

```
structure PreStream (A : Type u) : Type (max u (v + 1)) where
  corec :: {Carr : Type v} (gen : Carr → A × Carr) (init : Carr)
def hd {A : Type u} (a : PreStream A) : A := (a.gen a.init).fst

def tl {A : Type u} (a : PreStream A) : PreStream A :=
  (a.gen, (a.gen a.init).snd)
```

Listing 10: PreStream and destructors

Then I defined a bisimilarity relation; heads being equal and tails bisimilar as seen in Section 2.6. I proved this was an equivalence relation (reflexive, symmetric, transitive). Using this I defined a setoid on `PreStreams`.

```
def Bisim.refl (x : PreStream A) : Bisim A x x :=
  (Eq, (· ▯ .step rfl rfl), rfl)

def Bisim.symm (h : Bisim A a b) : Bisim A b a :=
  have ⟨r, his, rel⟩ := h
  ((fun v ⇒ r v ·), (match his · with | .step ha hb ⇒ .step ha hb.symm), rel)

def Bisim.trans (hab : Bisim A a b) (hbc : Bisim A b c) : Bisim A a c :=
  have ⟨rab, hisab, hab⟩ := hab; have ⟨rbc, hisbc, hbc⟩ := hbc
  ( (∃ b, rab · b ∧ rbc b ·),
    fun ⟨_, hax, hxb⟩ ⇒ match hisab hax, hisbc hxb with
    | .step hax e₁, .step hxb e₂ ⇒ .step ⟨_, hax, hxb⟩ (e₁.trans e₂),
    ⟨b, hab, hbc⟩ )
```

(a) Bisim definition

(b) Equivalence relation

Figure 14: PreStream setoid

The definition of an SME Stream is then preobjects quotiented by this setoid. Quotients are famously hard to work with¹⁷. When lifting the destructors from `PreStreams` to `Streams` I have to go through the lifting function of quotients. When lifting to a quotient one has to provide the lifting function, then a proof that the function is stable under the setoid. The corecursor was defined using the constructor for `PreStreams`, under the quotient constructor.

```
def hd : SStream A → A :=
  Quotient.lift PreStream.hd
  fun _ _ ⟨_, his, rab⟩ ⇒ match his rab with | .step _ h ⇒ h

def tl : SStream A → SStream A :=
  Quotient.lift (Quotient.mk _ · PreStream.tl)
  fun _ _ ⟨r, his, rab⟩ ⇒ match his rab with
  | .step h _ ⇒ Quotient.sound ⟨r, his, h⟩

def corec {A Carr : Type _}
  (gen : Carr → A × Carr)
  (init : Carr)
  : SStream A :=
  Quotient.mk _ (.corec gen init)
```

(a) Destructors for SStreams

(b) corec for Streams

Figure 15: corec and destructors for SStreams

The next step was defining a coinduction principle, in the code this is called `bisim` to align to convention with Mathlib [2]. The proof of this proceeds by quotient soundness and can be found in `SDefs.lean`.

¹⁷<https://proofassistants.stackexchange.com/questions/908/what-exactly-is-setoid-hell> <https://stackoverflow.com/questions/65493694/why-do-calculus-of-construction-based-languages-use-setoids-so-much/65509179>

```

coinductive Bisim
  (A : Type _)
  : SStream A
  → SStream A
  → Prop
| step
  {a b : SStream A}
  (cont : Bisim a.tl b.tl)
  (h : a.hd = b.hd)
  : Bisim A a b

theorem bisim {a b : SStream A} : Bisim A a b → a = b := by
  cases a, b using Quotient.ind₂; next a b ⇒
    rintro ⟨r, his, hhold⟩
    apply Quot.sound; change PreStream.Bisim A a b
    exact {
      fun x y ⇒ r (.mk _ x) (.mk _ y),
      fun {x y} hxy ⇒ match his hxy with
      | .step htl hhd ⇒ .step htl hhd,
      hhold
    }

```

(a) Bisim definition

(b) Coinduction principle

Figure 16: Coinduction principle for SStreams

3.2.2 Progressive approximation streams

sme/Sme/Stream/PDefs.lean

To implement PA streams. First I implement the stream's base functor. Streams have one constructor (so a head of 1), and each of the families only hold one instance of the value (so child $\lambda i () \mapsto 1$).

```

def PStream.Base : MvPFunctor 2 :=
  (PUnit, fun _ _ ⇒ PUnit)
def PStream (A : Type u) :=
  MvPFunctor.M PStream.Base fun _ _ ⇒ A

```

(a) Functor

(b) PStream

Figure 17: PStream definition

From here I defined the destructors of streams by calling the child with the correct indicies.

```

def hd (x : PStream A) : A :=
  x.dest.snd (.fs .fz) .unit
def tl (x : PStream A) : PStream A :=
  x.dest.snd .fz .unit

```

(a) Head of a PStream

(b) Tail of a PStream

Figure 18: PStream destructors

For symmetry I defined a syntactically identical bisimilarity relation on PA streams, and for this I prove a coinduction principle for PStreams of this relation. This proof proceeded by using the coinduction principle on general polynomials.

```

coinductive Bisim
  (A : Type _)
  : PStream A
  → PStream A
  → Prop
| step {a b : PStream A}
  (cont : Bisim a.tl b.tl)
  (h : a.hd = b.hd)
  : Bisim A a b

theorem bisim {a b} : Bisim A a b → a = b := by
  refine fun ⟨r, his, holds⟩ ⇒ MvPFunctor.M.bisim _ r
  (fun a b rab ⇒ ⟨.unit, fun | .fz, .unit ⇒ hd a, fun .unit ⇒ tl a, ?_⟩) a b holds
  have ⟨htl, hhd⟩ := his rab
  refine ⟨fun .unit ⇒ tl b, ?_, ?_, fun .unit ⇒ htl⟩
  <|> refine Sigma.ext rfl <| heq_of_eq <| funext₂ fun | .fz, .unit | .fs .fz, .unit ⇒ ?_
  any_goals rfl
  change hd b = hd a
  exact hhd.symm

```

(a) Bisimilarity of PStreams

(b) Coinduction principle for PStreams

Figure 19: Coinduction principle for PStreams

The corecursor was lifted from the definition on M -types.

```

def corec {A Gen : Type _}
  (gen : Gen → A × Gen)
  : Gen → PStream A :=
  .corecU _ (fun g ⇒ ⟨.up .unit, fun
  | .fz, _ .up .unit ⇒ .up (gen g).snd
  | .fs .fz, .up .unit ⇒ .up (gen g).fst⟩)

```

Listing 11: corec for PStreams

3.2.3 Proving the equivalence

sme/Sme/Stream/Equiv.lean

SM2

The functions of this equivalence are corecursors parameterised by the destructors of the opposite type. Proving that these are inverses was involved. I toyed around with a few different relations trying to make it work. In the end I landed on the straightforward equality as seen in Figure 20a. This solved and made the entire proof small after cleaning.

Having done this I was ready to approach the case of the polynomial. This turned out to be harder than I expected. Observant readers may notice the statement I proved is subtly the wrong statement. At the time I did not see this, but the equivalence should not be $\text{SStream}\{u, u\} A \approx \text{PStream } A$, but $\text{SStream}\{u, \max u \mathcal{V}\} A \approx \text{PStream } A$. This statement is harder to prove.

```
def Sme.Stream.equiv.{u} {A} : SStream.{u, u} A = PStream A where
  toFun := PStream.corec SStream.dest; invFun := SStream.corec PStream.dest
  left_inv _x := SStream.bisim (
    fun a b => a = .corec PStream.dest (.corec SStream.dest b),
    fun {a _b} (hab : a = _) => hab ▯ .step rfl rfl,
    rfl )
  right_inv _x := PStream.bisim (
    fun a b => a = .corec SStream.dest (.corec PStream.dest b),
    fun {a _b} (hab : a = _) => hab ▯ .step rfl rfl,
    rfl )
```

(a) Equivalence

Figure 20: Key code involved in Stream equivalence

3.3 State-machine encoding of M -types

sme/Sme/M/*

SM3, SM5

Recalling the definition of the SME from Section 2.5.4, and proceeding how we did in Section 3.2.1. First make a class of preobjects, then define a bisimilarity relation, make the SME by quotienting this relation.

3.3.1 Rephrasing bisimilarity for M -types

SS1

I implemented this dissertation using the definition of bisimilarity as given in Mathlib [2] as Figure 21a. This definition turned out to be hard to work with as it introduced heterogeneous equalities, and required unfolding the definition. Later in my dissertation I discovered an equivalent formulation using the universal property of the terminal object, since bisimilarity requires all but the last family to be equal, I equalize last families. This reduces the up-front cost of proving an M -type bisimilarity, as previously one had to instantiate a common head a , a shared child-family in all but the last argument f , and two families in the last argument f_1, f_2 . Finding these values was difficult and time consuming as they usually required unfolding the definition. The definition given in Figure 21b ended up being much easier to use. One of the reasons for this is that all the proofs about mappings of polynomials can be applied in a `simp` call to get the preliminary proof. This makes bisimilarity statements easier to work with.

<pre>def MvPFunc.M.bisim (R : P.M α → P.M α → Prop) (h : ∀ (x y : P.M α), R x y → ∃ a f f₁ f₂, M.dest P x = ⟨a, TypeVec.splitFun f f₁⟩ ∧ M.dest P y = ⟨a, TypeVec.splitFun f f₂⟩ ∧ ∀ (i : (P.B a).last), R (f₁ i) (f₂ i)) (x y : P.M α) (r : R x y) : x = y := ...</pre>	<pre>def bisim_map (R : P.M α → P.M α → Prop) (x : R a b) (h : ∀ s t, R s t → ∃ (x : (TypeVec.id :: Function.const _ PUnit.unit) <\$\$> s.dest = (TypeVec.id :: Function.const _ PUnit.unit) <\$\$> t.dest), ∀ v, R (s.dest.snd .fz v) <Δ t.dest.snd .fz <Δ cast (congr (congr rfl (Sigma.ext_iff.mp x).1) rfl) v) : a = b := ...</pre>
--	---

(a) Mathlib [2] definition

(b) New definition

Figure 21: Equivalent bisimilarity definitions

3.3.2 PreM

sme/Sme/M/PreM.lean

SM3

To define PreM, we proceed like we did for Streams. We have to use the definition `corecU` over `corec`.

```

/-- Preobjects for the SME encoded M-Type -/
@[pp_with_univ]
structure PreM (P : MvPFunctor.{u} (n + 1)) (α : TypeVec.{u} n) where
  corec ::
  {β : Type v}
  (gen : β → MvPFunctor.uLift.{u, v} P (TypeVec.uLift.{u, v} α :: ULift.{u, v} β))
  (g : β)

```

Listing 12: PreM definition

For the case of streams, we had two destructors `hd` and `tl`, in this case we have only one `dest`. Because of universe-levels we can not define a signature like `dest : PreM P α → P (α :: PreM P α)`, as `PreM P α` resides in **Type** $\max \mathcal{U}(\mathcal{V} + 1)$ for some $\mathcal{V}$, instead we get the signature `dest : PreM P α → ULift.{max  $\mathcal{U}$  ( $\mathcal{V} + 1$ )} P (ULift α :: PreM P α)`.

<pre> /-- Destructor of PreM types -/ @[inline, specialize P] def dest (x : PreM.{u, v} P α) : MvPFunctor.uLift P (TypeVec.uLift.{u, v + 1} α :: PreM.{u, v} P α) := map (TypeVec.id :: (PreM.corec x.gen .down)) < transliterate.{u, v, v+1} < x.gen x.g </pre>	<pre> @[simp] theorem dest_corec (gen : β → MvPFunctor.uLift.{u, v} P (TypeVec.uLift.{u, v} α :: ULift.{u, v} β)) (g : β) : (corec gen g).dest = (TypeVec.id :: (corec gen . ULift.down)) <\$\$> (transliterate.{u,v,v+1} < gen g) := rfl </pre>
(a) Destructor for PreM	(b) Equation for destructuring a corec

Figure 22: Destructor and β -rule for PreM

In addition to the two functions `corec` and `dest` we have defined for `PreM`, I also need `ULifting` of `PreMs`. This is a function taking a `PreM.{ $\mathcal{U}$ ,  $\mathcal{V}$ } P α` to a `PreM.{ $\mathcal{U}$ ,  $\max \mathcal{V}$   $\mathcal{W}$ } P α`.

```

@[pp_with_univ, inline]
def uLift (a : PreM.{u, v} P α) : PreM.{u, max v w} P α :=
  .corec (fun x =>
    have v := a.gen x.down
    (v.fst.transliterate, (.transliterate :: .up .transliterate) @ v.snd @ .transliterate)
  ) (ULift.up.{w} a.g)

```

Listing 13: ULifting of PreM

With all this plumbing done we can now move on to define bisimilarity. We use the definition of bisimulation from Section 3.3.1.

```

def IsBisim (R : PreM.{u, v} P α → PreM.{u, v} P α → Prop) :=
  ∀ s t, R s t →
  ∃ h : (TypeVec.id :: Function.const _ PUnit.unit) <$$> s.dest
  = (TypeVec.id :: Function.const _ PUnit.unit) <$$> t.dest,
  ∀ v, R (s.dest.snd .fz v) (t.dest.snd .fz (cast (by
    rw [show s.dest.fst = t.dest.fst from (Sigma.ext_iff.mp h).1]
  ) v))
def Bisim : PreM.{u, v} P α → PreM.{u, v} P α → Prop :=
  (∃ R, IsBisim R ∧ R . ·)

```

Listing 14: Bisimilarity of PreM types

We then had to prove it is an equivalence relation.

<pre> theorem refl (x : PreM.{u, v} P α) : Bisim x x := (Eq, fun _, _, .refl _ => (rfl, fun v => rfl), rfl) </pre>	<pre> theorem trans (hab : Bisim a b) (hbc : Bisim b c) : Bisim a c := have {r, hisAb, rab} := hab have {r₂, hisBc, rbc} := hbc ((fun a c => ∃ b, r, a b ∧ r₂ b c), fun x, z, ⟨y, rxy, ryz⟩ => (by have {hxy, hxyrel} := hisAb _ _ rxy have {hyz, hyzrel} := hisBc _ _ ryz refine {hxy.trans hyz, fun v => ⟨_, (hxyrel v), ?_⟩} have := (hyzrel < cast (by rw [show x.dest.fst = y.dest.fst from (Sigma.ext_iff.mp hxy).1] v) rw [cast_cast, cast_eq] at this exact this), ⟨b, rab, rbc⟩) </pre>
(a) Reflexivity	(b) Transitivity
<pre> theorem symm (h : Bisim a b) : Bisim b a := have {r, his, rab} := h ((fun a b => r b a), fun a b h => have {h, hrel} := his _ _ h {h.symm, fun v => by have := hrel < cast (by rw [show b.dest.fst = a.dest.fst from (Sigma.ext_iff.mp h).1] v rw [cast_cast, cast_eq] at this exact this) , rab) </pre>	
(c) Symmetry	

Figure 23: Equivalence relation for PreM bisimilarity.

This gives `PreM` a setoid structure and be used to instantiate the quotient.

3.3.3 SM

[UPSTREAMED], sme/Sme/{M/SM.lean, HEq.lean}

SM3

I expected implementing the SME M -types to go as smoothly as implementing SME streams. Defining the corecursor was as simple. Every other definition was much harder.

The destructor of SMs will be given by quotient lifting as for streams. The function is also similar: take the preobject destructor and equalize deeper occurrences as to not leak the type. The stability of this function is proven by soundness of quotients for the corecursive case, and lifting the equality from bisimilarity for the other cases.

Proving the coinduction principle difficult initially. The reason for this, is heterogeneous equalities. Since heterogeneous equalities have few congruences¹⁸, often I need to solve them on a case-by-case basis. As a consequence I ended up writing a few crucial lemmas, these include `dcongr_heq` which I upstreamed. Therefore, this proof has both forward and backward sections. To save our sanity, I decided to rewrite it using Section 3.3.1, after doing this the proof fell out.

```
theorem bisim : Bisim a b → a = b := by
  rintro ⟨r, his, hhold⟩
  cases a, b using Quotient.ind₂; apply Quot.sound; change PreM.Bisim _ _
  refine (fun x y ⇒ r (.mk _ x) (.mk _ y), fun s t rst ⇒ ?_ , hhold)
  clear *-his rst
  have ⟨hm, h⟩ := IsBisim.Alt_eq.mp his _ _ rst
  dsimp [dest] at hm h
  rw [MvFunctor.map_map, MvFunctor.map_map, TypeVec.appendFun_comp'] at hm
  exact ⟨hm, h⟩
```

Listing 15: Bisimilarity of SM-types

The next proof I had to do was with regards to universe lifting of SME M -types, this involves lifting the carrier instead of any part of the polynomial. This is the universe lifting function we defined for PreM types. Universe lifting of this component is needed for proving the generalized equivalence we stated in Section 3.2.3.

3.3.4 Proving the equivalence

sme/Sme/M/Equiv.lean

SM5

We know from streams, that each of the directions will be given by each of their corecursors. Further universe changes also must be handled. Finding the functions that satisfied these universe changes was non-trivial, as `map` can not change universe levels, therefore we need new functions for this (`transliterateMap`). The resulting functions are given in Listing 16, here we can see the subtle universe changes both up and down.

```
let msm := MvFunctor.map (TypeVec.id ::: ULift.up.{ℳ, max ℳ ℳ' + 1}) · SM.dest
let mpa := liftAppend.{ℳ, max ℳ ℳ'} · M.dest P · ULift.down.{max ℳ ℳ', ℳ}
let toFun := .corecU P msm,
let invFun := .corec mpa · ULift.up,
```

Listing 16: Functions of the equivalence

We now need to prove that the two functions are inverses. While proving these equalities I was not familiar with bisimilarities, and tried many invariants. Finally I landed on forcing their equalities. Once this was found, I had to prove it was a bisimilarity. I picked the head of one of the applications, and was lucky this was definitionally equal. This means the equality for the children is homogenous. The other direction followed analogously.

¹⁸An example is $(\text{List } A = \text{List } B) = (A = B)$ is independent of Lean.

```

let msm := MvFuncor.map (TypeVec.id :: ULift.up.{u, max u v + 1}) · SM.dest
let mpa := liftAppend.{u, max u v} · M.dest P · ULift.down.{max u v, u}
{
  toFun := .corecU P msm,
  invFun := .corec mpa · ULift.up,
  left_inv := fun x => SM.bisim (
    (· = (.corec mpa · ULift.up · .corecU _ msm) ·),
    by
      rintro _ <⟨gen, g⟩> rfl
      exact (MvPFuncor.liftR_iff _ _).mpr (
        .up (gen g).fst.down,
        fun
          | .fz, h => .corec mpa
            < ULift.up
            < M.corecU P msm
            < corec gen (((gen g).snd Fin2.fz · ULift.up) h.down).down
          | .fs s, .up h => (gen g).snd (.fs s) (.up h) ▷.down ▷ .up,
        fun
          | .fz, h => .corec gen ((gen g).snd .fz (.up h.down)).down
          | .fs s, h => (gen g).snd (.fs s) (.up h.down) ▷.down ▷ .up,
        Sigma.ext rfl < heq_of_eq < funext fun | .fz | .fs _ => rfl,
        Sigma.ext rfl < heq_of_eq < funext fun | .fz | .fs _ => rfl,
        fun | .fz, h | .fs _, h => rfl,
      ),
      rfl
    ),
  right_inv := fun x => M.bisim _
    (· = (.corecU _ msm · .corec mpa · .up) ·)
    (fun a b => (· ▯ (
      (corec mpa < .up b).dest.fst.down,
      ((M.dest P b).snd ·.fs),
      (M.corecU P msm < (corec mpa < .up b).dest.snd Fin2.fz < .up ·),
      (M.dest P b).snd .fz,
      Sigma.ext rfl < heq_of_eq < funext fun | .fz | .fs _ => rfl,
      Sigma.ext rfl < .refl _,
      fun _ => rfl
    )))
  -rfl,
}

/-- info: 'Sme.SM.equivP' depends on axioms: [propext, Classical.choice, Quot.sound] -/

```

Listing 17: SME and PA equivlance equivP

3.4 The Alternative Representation (AltRepr) Type

sme/Sme/AltRepr.lean

AM1, AM2, AM3

Mathlib [2] has a type called `Shrink`. This is a type which takes an equivalence between an $A : \text{Type } \mathcal{U}$ and a $B : \text{Type } \mathcal{V}$, and returns a $\text{Type } \mathcal{U}$. It has an equivalence $B \approx \text{Shrink } A \ B$. The definition given in Mathlib [2] is non-computable using choice, this makes it useless for code-generation.

```

class Small (α : Type ℳ) : Prop where equiv_small : ∃ S : Type ℳ, Nonempty (α = S)

def Shrink (α : Type ℳ) [Small.{ℳ} α] : Type ℳ := Classical.choose (@Small.equiv_small α _)

@[no_expose]
noncomputable def equivShrink (α : Type ℳ) [Small.{ℳ} α] : α = Shrink α :=
  Nonempty.some (Classical.choose_spec (@Small.equiv_small α _))

```

Listing 18: Mathlib [2] definition of Shrink

Originally I tried to make it computable by adding an `@[implemented_by]` to it, which turned out not to work. The reason this did not follow from `Shrink` strangely satisfies univalence: given `Shrink A` and `Shrink B` along with $A \approx B$, one can prove `Shrink A = Shrink B`. Aaron Liu from the Mathlib [2] community used this in combination with quotients and `trustCompiler` to prove false. From this I understood that I would have to make my own variation of the type. The proof Aaron Liu used can be found in Section 6.D.

From this I decided to make my own variant I called the `AltRepr` type, as it gave a type an alternative representation. Given an isomorphism $\text{eq} : \alpha \approx \beta$ for some types $\alpha : \text{Type } \mathcal{U}$ and $\beta : \text{Type } \mathcal{U}$, I define the type `AltRepr α β eq` to have two constructor-eliminator pairs `mkA`, `mkB`, `destA` and `destB`, satisfying the diagram seen in Figure 24a. Additionally I had an elimination principle satisfying expected β -rules with the two constructors. Through making the definition opaque I had problems with universe levels.

I ended up solving this using a transliteration. This let me eliminate into any universe independent of $\mathcal{U}$ and $\mathcal{V}$.

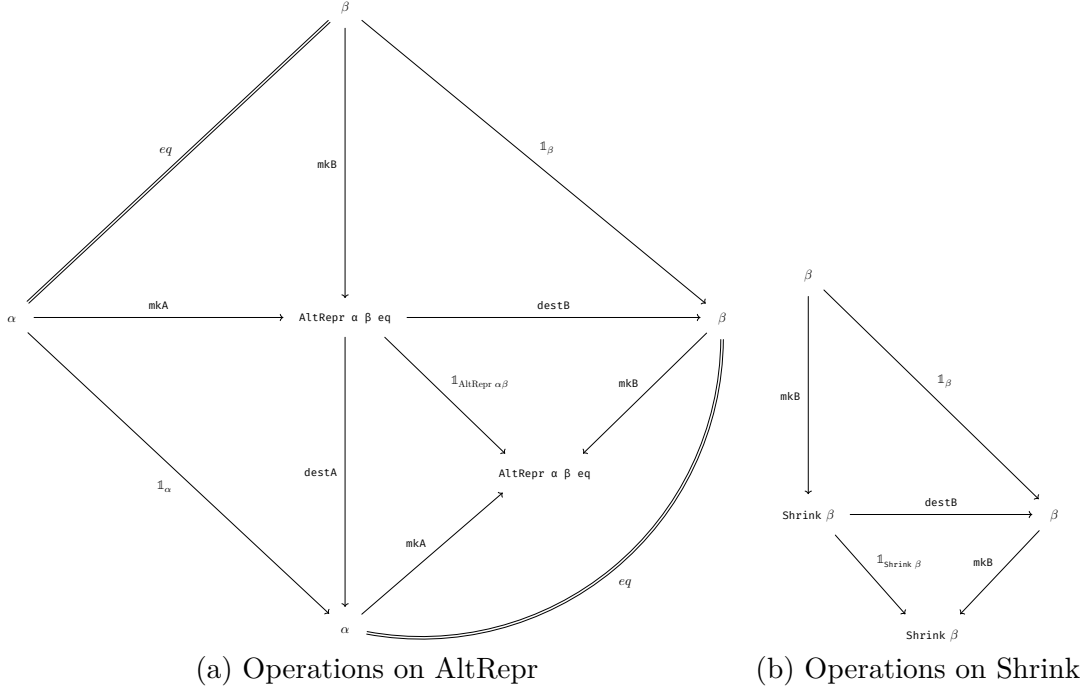


Figure 24: Universe transforming types.

3.4.1 High performance Low universe M -type (HpLuM)

sme/Sme/M/HpLuM.lean

SC1

After implementing the `AltRepr` type, it is time to put it to use. To do this, first I tried to use the equivalence, what I found was this leaked the universe of the carrier. I realised, I did not have to make a new equivalence for this, but paste the previous equivalence with itself. I called this “equiv cross universe” ($\text{equivXU} : \text{SM}.\{\mathcal{U}, \max \mathcal{U} \mathcal{V}\} P \alpha \approx \text{SM}.\{\mathcal{U}, \max \mathcal{U} \mathcal{W}\} P \alpha$). One can observe this equivalence forms a transliteration. Additionally one might wonder, does composing the equivalence with itself become a no-op? To which the answer is no, as we are parameterised in universe levels, so technically these are different functions. This became a challenge as the performance of this equivalence was $\mathcal{O}(n^2)$. To avoid this, I decided to mark this as `@[implemented_by, irreducible]` with a cast. Once this was done I could start implementing the `HpLuM`.

The `HpLuM` is the `AltRepr` type, parameterised by the equivalence given in the last section at universes $\mathcal{U}, \mathcal{U}$. These universes prevent the carrier from being leaked. From here we define the corecursor of the same signature as in the `SM`, this uses a cocktail of universe changes, first one up using `SM.uLift`, then one down using `equivXU`, finally being passed into a `mkB` constructor of the `AltRepr` type. The next step is giving a `dest : HPLuM P alpha → P (alpha :: HPLuM P alpha)`, this is done using the eliminator for the `AltRepr`-type. The reason I use this instead of a `destB` (which is what `rec` compiles to), using the eliminator lets me prove the coherence of the SME and PA inline.

Once this was done I started proving equations of these methods. The first one was `dest_corec`, which has the expected signature. Followed by a coinductive principle. I also added an in-universe version of `corec` called `corec'`, this has the exact behaviour of the old non generalized corecursor on PA M -types. In addition to this I added corecursor unrolling lemmas and mapping lemmas. From here I then proved co-Lambek’s theorem [10]. Further I proved `HpLuM` is multifunctorial as the polynomial definition, along with destructor lemmas for `map`.

I added functionality to reason about polynomial equivalents (Section 3.1.3), this is the reason I picked the CTF to be the outParam, as it lets these functions synthesize the correct CTF without annotations.

I also found I needed a definition of ulifting of HpLuMs. Like everything to do with universe levels, this turned out to be a challenge. I also proved naturality conditions of these.

The next thing I had to prove, was transportation of natural isomorphisms. This takes a natural isomorphism between P and P' , and lifts it to a natural isomorphism between $\text{HpLuM } P$ and $\text{HpLuM } P'$. This will be used when we start working with futumorphic productivity.

With this completed, I have a usable M -type library.

3.5 Non-Termination Monad (NTMonad)

sme/Sme/ITree/*

NM1

The NTMonad is an example of a coinductive datatype. It has two constructors $\text{tau} : \text{NTMonad } A \rightarrow \text{NTMonad } A$ and $\text{val} : A \rightarrow \text{NTMonad } A$. The values of this type will always be either some value of taus then a val , or an a unique value $\text{spin} = \text{tau spin}$. One can think of these as *coroutines*, where interlacing destructing of taus , is interlacing computation.

I implemented them by taking the cofixpoint of the Sum functor. Mainly I ended up focusing on ITrees as they are a strict generalization of the NTMonad, I still implemented the NTMonad as an exercise. This is adequate as $\text{NTMonad } A = \text{ITree EmptyE } A$, so it would be a waste to double up the implementation.

3.6 Interaction Trees

sme/Sme/ITree/*

IM1, IM2, IM4, IS2

To define ITrees in lean, we implement the functor, and prove results about polynomial equivalents on them. When KTrees are mentioned in the ITree paper [1], it is a function of the form $A \rightarrow \text{ITree } E \rightarrow B$, for those familiar these are Kleisli morphisms [19].

<pre> CoInductive itree (E: Type -> Type) (R: Type): Type := Ret (r: R) Tau (t: itree E R) Vis {A: Type} (e : E A) (k : A -> itree E R) </pre> (a) Rocq definition from [1]	<pre> @[grind] inductive Base (p R : Type _) tau (r : p) ret (t : R) vis {A : Type} u (e : E A) (k : A -> p) inductive PHBase : Type max v (u + 1) tau ret vis (A : Type u) (e : E A) def PBase : MvPFunctor.{max v (u + 1)} 2 where A := PHBase E B .ret => !![PUnit, PEmpty] .tau => !![PEmpty, PUnit] .vis A _ => !![PEmpty, ULift.{max v (u + 1), u} A] </pre> (b) Parts of the Lean definition
--	--

Figure 25: Defintions of ITrees

ITrees have many operations and equations defined in the paper [1]. These can be seen in Table 1, here I have annotated all of the functions I have implemented (as L) from the paper [1] (as R), and the other implementation of ITrees for lean by M. Sammler [3]. A boon of the Lean implementations over Rocq, is that strong bisimilarity implies equality; the coinduction principle is provable in Lean. This fact is inherited from the M -type definition having a provable coinduction principle. This makes using some of the proofs easier.

Interaction Tree Operations • $\text{ITree } E \ A : \text{Type}^{\text{LRM}}$ • $\text{tau} : \text{ITree } E \ A \rightarrow \text{ITree } E \ A^{\text{LRM}}$ • $\text{ret} : A \rightarrow \text{ITree } E \ A^{\text{LRM}}$ • $\text{vis } \{R\} : E \ R \rightarrow (R \rightarrow \text{ITree } E \ A) \rightarrow \text{ITree } E \ A^{\text{LRM}}$ • $\text{bind} : \text{ITree } E \ A \rightarrow (A \rightarrow \text{ITree } E \ B) \rightarrow \text{ITree } E \ B^{\text{LRM}}$ • $\text{trigger} : E \ A \rightarrow \text{ITree } E \ A^{\text{LR}}$	Hetrogenous weak bisimilarity • $\text{eutt } (r : A \rightarrow B \rightarrow \text{Prop}) : \text{ITree } E \ A \rightarrow \text{ITree } E \ B \rightarrow \text{Prop}^R$
Events and subevents • $(e : E \ R)^{\text{LRM}}$ R is the result of event e • $E \ +' \ F^{\text{LRM}}$ disjoint union of effects • $\text{class } E \prec F^{\text{LRM}}$ E is a subevent of F • $\text{trigger } [E \prec F] : E \rightsquigarrow \text{ITree } F^R$	Strong and weak bisimilarity • $_ \equiv _ : \text{ITree } E \ A \rightarrow \text{ITree } E \ B \rightarrow \text{Prop}^{\text{LRM}}$ • $_ \approx _ : \text{ITree } E \ A \rightarrow \text{ITree } E \ B \rightarrow \text{Prop}^{\text{LR}}$ • $\forall a \ b. a \equiv b \rightarrow a = b^{\text{LM}}$
Parametric functions $E \rightsquigarrow F := \forall X, E \ X \rightarrow F \ X^{\text{LR}}$	Monadic interpretation $\text{interp } [\text{Monad } M] [\text{MonadIter } M] : (E \rightsquigarrow M) \rightarrow (\text{ITree } E \rightsquigarrow M)^{\text{LRM}}$

Table 1: Functions implemented in the dissertaton (L)ean, in (R)ocq [1] and (M). Sammler [3]

Structural Laws • $\text{tau } t \approx t^{\text{LR}}$ • $t \approx \text{spin} \rightarrow t = \text{spin}^L$ • $\text{bind } (\text{tau } t) \ k = \text{tau } (\text{bind } t \ k)^{\text{LR}^*M}$ • $\text{bind } (\text{vis } e \ k_1) \ k_2 = \text{vis } e \ \lambda y \mapsto \text{bind } (k_1 \ y) \ k_2^{\text{LR}^*M}$	Congruences • $t_1 \approx t_2 \rightarrow \text{tau } t_1 \approx \text{tau } t_2^{\text{LR}}$ • $k_1 \approx k_2 \rightarrow \text{vis } e \ k_1 \approx \text{vis } e \ k_2^{\text{LR}}$ • $t_1 \approx t_2 \rightarrow k_1 \approx k_2 \rightarrow \text{bind } t_1 \ k_2 \approx \text{bind } t_2 \ k_2^{\text{LR}}$ • $k_1 \approx k_2 \rightarrow \text{iter } k_2 \ v \approx \text{iter } k_2 \ v^{\text{LR}^\dagger}$
Equivalence relation • $a \approx a^{\text{LR}}$ • $a \approx b \rightarrow b \approx a^{\text{LR}}$ • $a \approx b \rightarrow b \approx c \rightarrow a \approx c^{\text{LR}}$	Monad Laws • $(\text{ret } v) \gg= k = k \ v^{\text{LR}^*M}$ • $t \gg= \text{ret} = t^{\text{LR}^*M}$ • $x \gg= f \gg= g = x \gg= (f \cdot \gg= g)^{\text{LR}^*M}$

Table 2: Equations implemented in the dissertaton (L)ean, in (R)ocq [1] and (M). Sammler [3]¹⁹

3.6.1 The ITree Monad

sme/Sme/ITree/Monad.lean

IM3, IS1, IS3

ITrees form a monad, where the binding operation corresponds to pasting trees together, as seen in Figure 26; `bind` iterates through the tree and calls the function on seeing a `ret`. The `ret` constructor is the monads unit. To ensure it is lawful there is a set of proof obligations, these can be seen in Table 2.

¹⁹The annotations in the table are as follows:

* = strong bisimilarity rather than equality in Rocq

† = proven using coinduction-up-to in Rocq

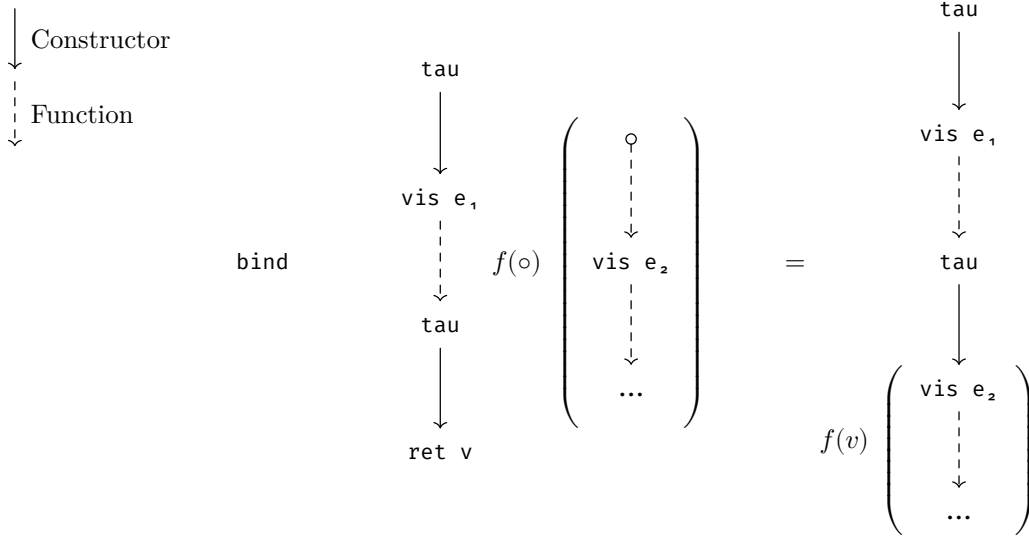


Figure 26: Visual representation of bind on ITrees

3.6.2 Weak bisimilarity

sme/Sme/ITree/WBisim/*.lean

IC1

Bisimilarity is often not sufficient for proving program equivalences. This comes from the fact that sometimes we care about more than equality. For example the program `println 10` and `println (10 + 0)` are *equivalent*, but not syntactically equal. We can note that the effects produced by these programs are the same, yet one takes more steps to get there. This is where we consider weak bisimilarity instead. One can think of weak bisimilarity is an equivalence relation modulo silent steps; two objects with the same observable effects but with a different tau count. The Rocq definition²⁰ and my definition differs. This is mainly because I decided to implement only homogeneous weak bisimilarity, and in Rocq they have coinduction-up-to. Coinduction-up-to is a technique where the prover does not have to provide the full bisimulation, but can rather use theorems to extend it [20], [21]. In my definition I need to ‘bake in’ as many principles as possible. This is the reason for the `refl` constructor. In the definition I use `EStep` to mean some amount of taus, and `Step` to mean a productive step to some non-tau value. `EStep` forms a linear join-semilattice, `Step` inherits some of this structure, but is easier to work with. One of the key pieces of structure helping this be closer to coinduction-up-to is the ability to skip taus in a proof, this is done using the principle `skip : EStep a c → EStep b d → Invariant E A R c d → Invariant E A R a b`, which operates directly on the invariant for the bisimilarity. This solves problems similarly to how it is done in the paper, it gives a principle where we can simplify equations by shifting them along the taus.

```
CoInductive eutf : itree E A → itree E B → Prop :=
| EqRet a b (REL: r a b) : eutf (Ret a) (Ret b)
| EqVis {R} (e : E R) k1 k2 (REL: ∀v, eutf (k1 v) (k2 v)) : eutf
(Vis e k1) (Vis e k2)
| EqTau t1 t2 (REL: eutf t1 t2) : eutf (Tau t1) (Tau t2)
| EqTauL t1 ot2 (REL: eutf t1 ot2) : eutf (Tau t1) ot2
| EqTauR ot1 t2 (REL: eutf ot1 t2) : eutf ot1 (Tau t2).
```

(a) Rocq definition

```
coinductive WBisim'
(E : Type u → Type u) (A : Type v)
: Base E (ITree E A) A → Base E (ITree E A) A → Prop
where
-- This case only exists for refl spin spin,

-- in fact it could be this,
-- but that would be harder to prove things about
| refl {a b c} : EStep a c → EStep b c → WBisim' E A a b
| ret {a b v} : Step a (.ret v)
→ Step b (.ret v) → WBisim' E A a b
| vis {V e a b a' b'}
```

(b) Lean definition

Figure 27: Weak-bisimilarity for ITrees

In the ITree paper [1] it is proven weak-bisimilarity is an equivalence relation. A table of what is proven in each implementation can be found in Table 2. Out of these the ones marked with † have been proven using coinduction-up-to. These were particularly technical to prove and required defining

²⁰The syntax is slightly simplified for reading purposes here

helper predicates. The main instance of this is `iter-congruence`, for this I had to define two predicates `IterTrace` and `IterCotrace` for productive and spinning `iters` respectively. This is used to define, if the two `KTrees` are weakly bisimilar, the generated `ITrees` have the same productivity behaviour. This proof was indirect and required the creative step of coming up with the (co)trace abstraction.

3.6.3 A formally verified optimization

`sme/Sme/ITree/WBisim/*.lean`

IC1

One of the main use cases of Interaction Trees is program verification. I implement a simple imperative language `IMP` as seen in Listing 19a, I give it the denotational semantic as Listing 19b, then I define an optimization `simpl` Listing 19c on this. All `simpl` does is simplifies constant expressions, the proof of this respecting the semantics is provided in the artifact, and is an example of how weak-bisimilarity can help prove an optimization correct.

```

inductive Imp (N : Type) : Type
-- arith
| add (a b : Imp N)
| mul (a b : Imp N)
| constN (n : Int)
-- variables
| var (v : N)
| assign (k : N) (a : Imp N)
-- IO
| print (a : Imp N)
| seq (a b : Imp N)
-- control
| ite (c t f : Imp N)
| while (c b : Imp N)
deriving DecidableEq, Repr

```

(a) Imp syntax

```

def sem {N} : Imp N → ITree (IOE @, MapE N Int)
Int
-- arith
| .add a b => do
  let a <- sem a
  let b <- sem b
  return a + b
| .mul a b => do
  let a <- sem a
  let b <- sem b
  return a * b
| .constN n => .ret n
-- var
| .var v => .trigger <- .inr <- .getD v 0
| .assign k v => do
  let v <- sem v
  let _ <- .trigger <- .inr <- .insert k v
  return 0
-- effects
| .print v => do
  let v <- sem v
  let _ <- .trigger <- .inl <- .print v
  return 0
| .seq a b => do
  let _ <- sem a
  sem b
-- control
| .while c b => .iter (fun _ => do
  let c <- sem c
  if c ≠ 0 then
    let _ <- sem b
    return .inl .unit
  else
    return .inr 0
) Unit.unit
| .ite c t f => do
  let c <- sem c
  if c ≠ 0 then sem t
  else sem f

```

(b) Imp semantics

```

def simpl {N} (inp : Imp N) : Imp N :=
  flat <- pass inp
where
  flat : Int @ _ → _
  | .inl v => .constN v
  | .inr v => v
  pass : _ → Int @ _
  | .add a b =>
    match pass a, pass b with
    | .inl a, .inl b => .inl (a + b)
    | .inr a, b => .inr <- .add a (flat b)
    | .inl a, .inr b =>
      .inr <- .add (.constN a) b
  | .mul a b =>
    match pass a, pass b with
    | .inl a, .inl b => .inl (a * b)
    | .inr a, b => .inr <- .mul a (flat b)
    | .inl a, .inr b =>
      .inr <- .mul (.constN a) b
  | .constN n => .inl n
  | .print v =>
    .inr <- .print <- flat (pass v)
  | .var n => .inr <- .var n
  | .assign n v =>
    .inr <- .assign n <- flat (pass v)
  | .seq a b =>
    match pass a, pass b with
    | .inl _, b => b
    | .inr a, .inl b =>
      .inr <- .seq a (.constN b)
    | .inr a, .inr b => .inr <- .seq a b
  | .ite c t f =>
    match pass c, pass t, pass f with
    | .inl 0, _, f => f
    | .inl (.ofNat (_ + 1)), t, _
    | .inl (.negSucc _), t, _ => t
    | .inr c, t, f =>
      .inr <- .ite c (flat t) (flat f)
  | .while c b =>
    match pass c, pass b with
    | .inl 0, _ => .inl 0
    | .inl (.ofNat (_ + 1)), b
    | .inl (.negSucc _), b =>
      .inr <- .while (.constN 1) <- flat b
    | .inr c, b =>
      .inr <- .while c <- flat b

```

(c) simpl

Listing 19: Imp and simpl

3.7 Free Monad

Originally when considering the choice points for creating a variable layer polynomial, I constructed it directly. This structure became known as `DeepThunks DT P (α :: β)`, and was given as the cofix-point of $\mu ((P \alpha -) \circ (\text{Sum } \beta -))$ (given in `sme/Sme/M/DT/Defs.lean`). Using this it `DeepThunks` it was possible to define a function `dtcorec {β} : (β → DT P (α :: β)) → β → M P α`, an injector `inject β : M P α → DT P (α :: β)`, and a flattener `flatten : DT P (α :: M P α) → M P α`. These

were extremely challenging to reason about, but in the end I successfully proved an unfolding lemma of the corecursor, where it corresponds to doing a mapping operation then flattening. As this seemed like an interesting structure, I scoured the literature for similar objects.

Through this I found futumorphism. These use the free monad to make a morphism $\text{futu } \{\beta\} : (\beta \rightarrow P (\alpha :: \text{Free } P \alpha \beta)) \rightarrow M P \alpha$. The relationship between the `Free` monad and `DT` can be seen in Figure 28²¹. This demonstrates that $P (\alpha :: \text{Free } \alpha \beta)$ is exactly `DT`. A crucial difference is that `Free` has a well-defined notion of constructors and eliminator, while `DT` has nothing equivalent. This means as I wrote around 1700 lines for `DT`, almost all this functionality fit into 700 lines for the futumorphism. The implemented functions can be seen in Table 3.

$$\underbrace{(P\alpha -) \triangleleft (\beta \oplus -) \triangleleft (P\alpha -) \triangleleft (\beta \oplus -) \triangleleft \dots}_{\text{DeepThunk}} \quad \overbrace{\hspace{10em}}^{\text{Free monad}}$$

Figure 28: Relationship between `Free` monad and `DT`

The definition given for deep-thunks used the carrier as one of the mapped parameters to the polynomial. Meaning I did not need to implement a dedicated map function, this turned out to be a mistake. The problem here is that any multivariate map can change the carrier. This meant that for all equations involving the multivariate map, had to be restricted to functions concatenated together. This also meant that working with binds became a mess for universe levels. Fundamentally we want to treat this as a different parameter so we can also use `do` notation. Proving facts about binds also became extremely tedious. We also did not have a monad unit as the structure of the base polynomial `P` can be arbitrary. Another problem was `DeepThunks` across universes required applying a sequence of natural transformations. These natural transformations had few equations other than naturality proven about them. Additionally some proofs are unproven as they became too difficult.

When implementing the `Free` monad, I applied each of these learnings. The definition used had the universe lifting built in, meaning we only had to reason about switching universes once (`dest_corec`). The carrier was also taken as a function parameter used to instantiate a constant. This solved the problem of maps being able to change the carrier, and at the same time allowed the free monad to satisfy the signature needed for the `Monad : (Type → Type) → Type` typeclass. This was also made even simpler by implementing the corecursor and the destructor using the standard Lean `Sums`. With these design decisions, implementing constructors (`recall` and `cont`) for the `Free` monad was just following the type signature. I added a third constructor `cont'` which was in-universe allowing for some definitions to be simplified. For both of these I added `cases` principles for the constructor sets. I also set up the corecursor to operate on `Sums` as well which made writing functions easy. The coinductive principle for `M`-types could also be specialized to the `Free` monad using relational lifting on `Sum` types. Building on the preexisting `simp` set for `Sums`, and marking the β -rules as `@[simp]`, proving many theorems became as simple as just invoking `simp`. The best examples of these can be seen in the mapping equations on constructors. These theorems were much harder to prove for `DeepThunks`. Since we are no longer relying multivariate maps to change the carrier, we now need a new map definition.

²¹Here $\triangleleft$ is application as seen in OCaml.

Free Monad Operations

- `Free P α β` : **Type**
- `recall` : $\beta \rightarrow \text{Free } P \alpha \beta$
- `cont'` : $P (\alpha :: \text{Free } P \alpha \beta) \rightarrow \text{Free } P \alpha \beta^*$
- `dest'` : $\text{Free } P \alpha \beta \rightarrow \beta \oplus P (\alpha :: \text{Free } P \alpha \beta)^*$
- `corec'` : $(\gamma \rightarrow \beta \oplus P (\alpha :: \gamma)) \rightarrow \gamma \rightarrow \text{Free } P \alpha \beta^*$
- `map` : $\text{Free } P \alpha \beta \rightarrow (\beta \rightarrow \gamma) \rightarrow \text{Free } P \alpha \gamma$
- `bind` : $\text{Free } P \alpha \beta \rightarrow (\beta \rightarrow \text{Free } P \alpha \gamma) \rightarrow \text{Free } P \alpha \gamma$
- `inject β` : $\text{HpLuM } P \alpha \rightarrow \text{Free } P \alpha \beta$
- `flatten` : $\text{Free } P \alpha (\text{HpLuM } P \alpha) \rightarrow \text{HpLuM } P \alpha$
- `futu'` : $(\beta \rightarrow P (\alpha :: \text{Free } P \alpha \beta)) \rightarrow \beta \rightarrow \text{HpLuM } P \alpha^*$

Monad Laws

- $(\text{recall } v) \gg= k = k \ v$
- $t \gg= \text{recall} = t$
- $x \gg= f \gg= g = x \gg= (f \cdot \gg= g)$

Functor Laws

- $\text{id} \langle \$ \rangle x = x$
- $(f \circ g) \langle \$ \rangle x = f \langle \$ \rangle g \langle \$ \rangle g$

Table 3: Functions implemented for the Free monad²²

²²The annotations in the table are as follows:

* = Cross universe version also implemented.

Chapter 4: Evaluation

This chapter will discuss and evaluate each of the components of the dissertation. As mentioned in Section 2.9, I have been writing proofs verifying the correctness of my different components. I refer to these throughout this chapter. Type signatures alone do not encode performance guarantees; therefore, I write benchmarks and read LCNF to evaluate these features. There will be a comparison of coinductive and ITree implementations, and finally an assessment of the ergonomics of futumorphic and deep-thunk productivity.

An overview of all the requirements laid out can be viewed in Table 4.

Req.	Met	Evidence	Path	Req.	Met	Evidence	Path
SM1	Y	Section 3.2	Stream/ SDefs.lean	NM1	Y	Section 3.3	NTMonad/ Defs.lean
SM2	Y	Section 3.2.3	Stream/ Equiv.lean	NM2	Y*	Section 4.4	ITree/ Monad.lean
SM3	Y	Section 3.3.3	M/Defs.lean	NS1	Y*	Section 4.4	ITree/ Monad.lean
SM4	Y	Section 4.1.1	-				
SM5	Y	Section 3.3.4	M/Equiv.lean	NC1	Y	TODO	NTMonad/ Defs.lean
SM6	Y	Section 3.5	NTMonad/ Defs.lean			sec:futu:case	
						(a) NT Monad	
			M/ {HpLuM,SM}.lean				
SS1	Y	Section 3.4.1					
SC1	Y	Section 3.4.1	M/HpLuM.lean				
SC2	Y	Section 3.6	ITree/Defs.lean				
			M/{DT/ *,Futu.lean}				
SC3	Y	Section 3.7					
SC4	N	Section 4.1.1	-				
(b) State-machine encoding				(c) AltRepr Type			

Table 4: Requirements and completion

4.1 State-machine encoding

For the state-machine encoding, there are multiple aspects to evaluate. For example performance, expressivity, and safety when compared to other implementations. The implementations I will test

are the PA encoding from Mathlib [2], a mathematically optimal encoding, and an implementation by M. Sammler [3] which was started during this project. Notably this final implementation is built on domain theoretic methods, meaning that instead of requiring productivity, all it requires is monotonicity. I will speak more about this when evaluating the futumorphic productivity.

4.1.1 Performance between SME and PA

After building the equivalence, and instantiating `AltRepr`, we now have the ability to compare the performance between 4 representations: The HpLuM implementation, the PA encoding, and the PreM and quotiented representations SM as theoretical optima. For SC4 to be the case, the HpLuM would have to be within 1σ of the PreM encoding.

I implemented streams for the different encodings, then I measured, using a monotonic clock, how long it took to destructure n -layers of the stream of natural numbers, I repeated this experiment 3 times. For the PA, I swept $n \in [0, 200]$, and for HpLuM and PreM, SM encodings $n \in [0, 5000]$. I fit polynomials for each of these implementations, then I plot samples, along with the fit. This generates the plot Figure 29. As we can see, there is a discrepancy between the HpLuM and PreM encodings. For an unknown reason the variance along each of these graphs pulsates, this effect is relatively minor, but unexplainable.

Reviewing the output plot we can see that the SME is $\mathcal{O}(1)$ under destructuring, as opposed to the PA encoding which is $\mathcal{O}(n)$. This is in line with the expected theoretical performance.

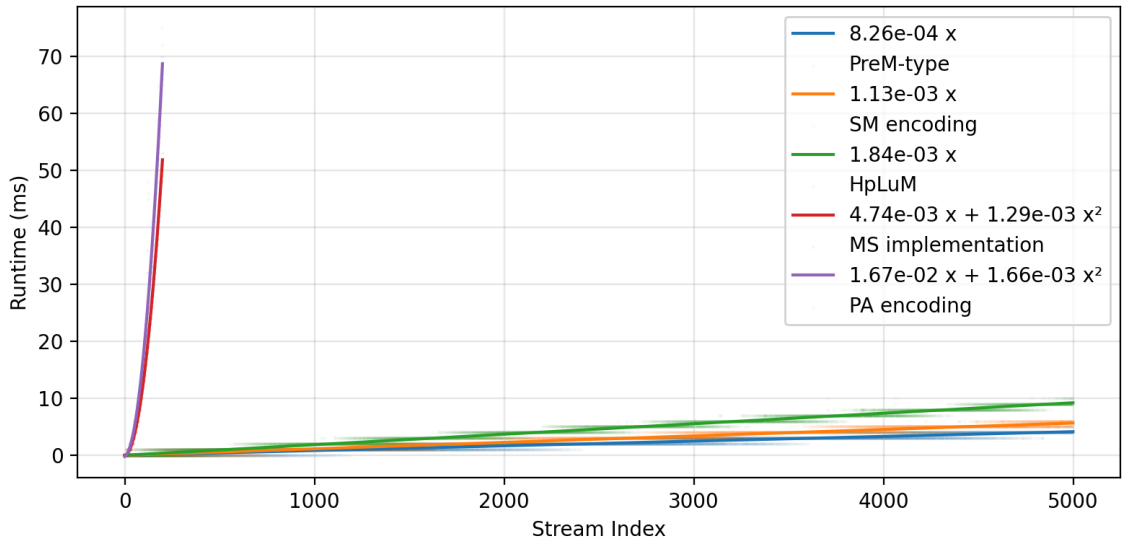


Figure 29: Performance of PA, HpLuM, and SM & PreM representation

The SM and PreM implementations are drawn from the same distribution. On the other hand, the HpLuM is 1.5x slower. The issue causing this has to do with the destructor function needing to do a universe lowering. This adds a fixed cost at each iteration, compared to the PreM which calls the destructor function.

4.1.2 Comparison of corecursor implementations

In this section, I compare the different implementations of coinductives across several aspects: performance, and ergonomics of object construction and usage.

	Diss.	Mathlib [2]	M. Sammler [3]
dest performance	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Type Macro	N	Y^{23}	N
Function definitions	futu	corec	partial_fixpoint

4.2 AltRepr Type

When it comes to the AltRepr type, we have it implemented AM1, and we have an eliminator AM2. We now have to assess whether or not it is zero cost AC1. To do this, rather than testing the performance, I inspect the generated code for each of the 5 functions. The 3 functions we care about are `destB` Listing 20a, `mkB` Listing 20b, and `rec` Listing 20c. Additionally, each of these are marked with an `@[inline]` hint, meaning that in compiled code they do not appear. As we see in each of these, the functions have become the identity. For the case of `mkA` Listing 20e and `destA` Listing 20d, they compile into the expected call to the equivalence. This also lets us verify that we have the behaviour of the type B AM3.

This means all criteria for the AltRepr Type are met.

<pre>[Compiler.result] size: 0 def AltRepr.destB A B eq a.1 : lCAny := return a.1</pre>	<pre>[Compiler.result] size: 0 def AltRepr.mkB A B eq a.1 : lCAny := return a.1</pre>	
(a) LCNF AltRepr.destB	(b) LCNF AltRepr.mkB	
<pre>[Compiler.result] size: 4 def AltRepr.destA A B eq a.1 : lCAny := let _x.2 := Equiv.symm._redArg eq; cases _x.2 : lCAny Equiv.mk toFun invFun left_inv right_inv => let _x.3 := toFun a.1; return _x.3</pre>	<pre>[Compiler.result] size: 3 def AltRepr.mkA A B eq a.1 : lCAny := cases eq : lCAny Equiv.mk toFun invFun left_inv right_inv => let _x.2 := toFun a.1; return _x.2</pre>	<pre>[Compiler.result] size: 1 def AltRepr.rec A B eq motive.1 hLog hCheap eqA eqB v : lCAny := let _x.2 := hCheap v; return _x.2</pre>
(d) LCNF AltRepr.destA	(e) LCNF AltRepr.mkA	(c) LCNF AltRepr.rec

Listing 20: LCNF for functions on the AltRepr Type

4.3 Interaction trees

For evaluating interaction trees, there are 3 main aspects to evaluate. These are function coverage, proof coverage, and performance. For performance, it is inherited directly from the performance of the *M*-Type, so I will focus on function and proof coverage.

Function coverage can be found in Table 1 and proof coverage Table 2. This is comparing against the ITree paper [1]. In private conversation with Nicolas Chappe, he informed me of some of the additions to the interaction tree library for Rocq. These include relations such as simulation up to taus (sutt), and relation up to taus (rutt). These relations turn out to be useful for compiler verification in for example compcert [22]. One of the reasons for this has to do with undefined behaviour in C, specifically non-termination. In C, a non-terminating function is UB and therefore should be able to be related to any other function. This is in line with the fact that the spinning ITree can be simulated by any other ITree. This is as opposed to how the spinning ITree is unique up to weak bisimulation.

On the other hand, Lean now has an ITree library, something multiple groups have requested. For this reason Tobias Grosser has expressed interest in using the current implementation for his project VeIR [23]. All positive requirements are met (IM1 IM2 IM3 IM4 IS1 IS2 IS3 IC1 IC2).

During the production of this dissertation, M. Sammler [3] also started work on an interaction tree library. This is done using his implementation of coinductives using domain theoretic means. This also

²³For Lean v4.25

means that his definition of `iter` is not guaranteed to be productive, but also has a nicer unfolding lemma. The reason for this has to do with trying to avoid weak bisimilarity, and therefore has no implementation of the relation. The primary difference between this dissertation and M. Sammler [3], is he has a much greater focus on effects and interpretation. This is something I explicitly avoided stated in IW1. From this regard both implementations have their merits.

4.4 Non Termination Monad

Once implementing the SME was done, I moved over to implementing the non-termination monad. Here I focused on getting as ergonomic an experience as possible using `mkE` and `destE` for polynomial equivalents. In doing this, I realised the expectation of ITrees being much harder was false. I then stopped working on the NTMonad after just implementing NM1, as NTMonad is a special case of ITrees with the empty event. I am counting NS1 and NM2 as completed, as the generalization encompasses it.

Later I came back to working on it for evaluating futumorphic productivity then completing NC1.

4.5 Futumorphic productivity

By inspecting the table Table 3, we can see we have implemented FM1 and FM2. In the code one can also find the cross universe futumorphism solving FC1. We also have theorems about flattening, mapping and injection proven in the code, these are all implemented to fill out a `simp`-set for futumorphism. The result of this is that when an end-user uses a futumorphism, all they have to provide is a mapping over constructor lemma, and the rest will automatically solve. The main theorems that make this possible, is the unfolding of futumorphism lemmas. For the case of `futu'`, the statement can be seen in Listing 21. With this implemented we have FS1. Furthermore, I added theorems stating `inject` is an injection, and `flatten` is `inject`'s left-inverse²⁴.

```
theorem futu'_flatten
  (f :  $\beta \rightarrow P$  ( $\alpha :: \text{Free } P \ \alpha \ \beta$ ))
  (seed :  $\beta$ )
  : (futu' f seed).dest
  = (TypeVec.id :: flatten  $\circ$  map.{u} {futu' f})
  <$$> f seed := by
  rw [futu'_flatten_mk, HPLuM.mk_dest]
```

Listing 21: Futumorphic unfolding lemma

4.5.1 Comparing function definitions

For evaluating the futumorphism, it is best to put practice over promise, for this we can look at some functions written using the current 3 possible styles of writing functions. These are directly using the corecursor and using a futumorphism. I also tried to compare it to M. Sammler [3], but the fixpoint turned out to try to compute the entire definition in finite memory, crashing my compiler, this means these functions would not be useful to compute with.

The main use of a futumorphism, is moving any recursive component of a corecursor out of the state domain, and into an explicit function. Doing this transformation lets the function be written exactly as one would expect to write a normal inductive function, just inserting corecursive calls as `recall` constructors. Three good examples of this can be seen in Listing 22, Listing 23 and Listing 24. This fact means that a productive function, is exactly a function who's generating function to the futumorphism is terminating.

²⁴These two are not equivalent statements.

<pre>def interlace_const (o : α) (v : SS α) : SS α := .corec' body (v, .true) where body : SS α $\times$ Bool $\rightarrow$ _ (v, .true) $\Rightarrow$ have (hd, tl) := v.dest SS.mk hd (tl, .false) (v, .false) $\Rightarrow$ SS.mk o (v, .true) /-- info: [1, 0, 2, 0, 3, 0, 4, 0, 5, 0] -/ #guard_msgs in #eval! (interlace_const 0 (nats 1)).take 10</pre> (a) Corec definition	<pre>def interlace_const' (o : α) : SS α $\rightarrow$ SS α := futu' body where body s := have (hd, tl) := s.dest SS.mk hd $\triangleleft$ cont' $\triangleleft$ SS.mk o $\triangleleft$ recall tl /-- info: [1, 0, 2, 0, 3, 0, 4, 0, 5, 0] -/ #guard_msgs in #eval! (interlace_const' 0 (nats 1)).take 10</pre> (b) Futu definition
---	--

Listing 22: Interlace constant

<pre>def stutter (v : SS α) : SS α := .corec' body (v, .true) where body : SS α $\times$ Bool $\rightarrow$ _ (v, .true) $\Rightarrow$ have (hd, _) := v.dest SS.mk hd (v, .false) (v, .false) $\Rightarrow$ have (hd, tl) := v.dest SS.mk hd (tl, .true) /-- info: [0, 0, 1, 1, 2, 2, 3, 3, 4, 4] -/ #guard_msgs in #eval! (stutter (nats 0)).take 10</pre> (a) Corec definition	<pre>def stutter' : SS α $\rightarrow$ SS α := futu' body where body s := have (hd, tl) := s.dest SS.mk hd $\triangleleft$ cont' $\triangleleft$ SS.mk hd $\triangleleft$ recall tl /-- info: [0, 0, 1, 1, 2, 2, 3, 3, 4, 4] -/ #guard_msgs in #eval! (stutter' (nats 0)).take 10</pre> (b) Futu definition
---	---

Listing 23: Stutter constant

<pre>def RLE_decode (s : SS (α $\times$ Nat)) : SS α := .corec' f s.dest where f ((v, 0), r) $\Rightarrow$ SS.mk v r.dest ((v, n+1), r) $\Rightarrow$ SS.mk v ((v, n), r) /-- info: [0, 1, 1, 2, 2, 2, 3, 3, 3, 3] -/ #guard_msgs in #eval! (RLE_decode ((fun .fz, a $\Rightarrow$ Prod.mk a a) <\$\$> nats 0)).take 10</pre> (a) Corec definition	<pre>def RLE_decode' : SS (α $\times$ Nat) $\rightarrow$ SS α := Free.futu' f where f x := have ((v, n), r) := x.dest iter v r n 0 $\Rightarrow$ SS.mk v $\triangleleft$.recall r n+1 $\Rightarrow$ SS.mk v $\triangleleft$.cont' $\triangleleft$ iter v r n /-- info: [0, 1, 1, 2, 2, 2, 3, 3, 3, 3] -/ #guard_msgs in #eval! (RLE_decode' ((fun .fz, a $\Rightarrow$ Prod.mk a a) <\$\$> nats 0)).take 10</pre> (b) Futu definition
--	---

Listing 24: Run-length encoding

Chapter 5: Conclusions

Coinductive datatypes are useful not only as a program verification tool, but also in general purpose computation. To this point Lean has only had implementations intended for theoretical reasoning, from work such as J. Avigad, M. Carneiro, and S. Hudon [6] and A. Keizer [5]. There continues to be interest in this topic on the Lean Zulip, and from other researchers with M. Sammler [3]. All prior work is slow, rendering it unusable for general purpose computation.

Lean now has a coinduction library with functions usable in general purpose computation. This means that we can now write, and verify non-terminating programs, and have the complexity we would like from the program. To aid in writing these functions, futumorphisms were implemented. These allow for mixed corecursive-recursive definitions to be written in a more natural way.

On top of this, this dissertation includes a feature rich interaction tree library. This has potential to be used directly in work that requires formally verified compilation or translation. T. Grosser has expressed interest in using this on extending VeIR [23]. In private conversation, M. Sammler is open to extensions to his ITree library for weak bisimilarity.

I have met all my success criteria, along with universe changing type, an ITree implementation and a futumorphism implementation.

5.1 Future work

5.1.1 Syntax macro for types

A. Keizer [5] puts forward a macro for constructing QPFs [6]. This is a required feature for any coinduction library, for this reason it would be highly desirable to have this. I have designed all the functions on `HpLuM` to be drop in replacements for `M`. This means it should be as simple as updating Alex's library to Lean 4.29, and updating the relevant occurrences.

5.1.2 Derecursifier for corecursive function

A more ambitious goal is finding a way to write a derecursifier for corecursive functions. This would be a macro taking a definition, and transforming a function body into a corecursive function. This would work similarly to `partial_fixpoint` but would need further work. One of the main reasons the futumorphism was implemented, was to serve as a middleground here, but it might even be possible to use it in the derecursifier.

5.1.3 Extending the ITree library

The main component of the ITree paper missing is the zoo of events, these would involve quite a bit of implementation.

Bibliography

- [1] L.-y. Xia *et al.*, “Interaction trees: representing recursive and impure programs in Coq,” *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019, doi: 10.1145/3371119.
- [2] The mathlib Community, “The Lean Mathematical Library,” in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, in CPP 2020. New Orleans, LA, USA: ACM, Jan. 2020. doi: 10.1145/3372885.3373824.
- [3] M. Sammler, “Coinductive.” <https://github.com/ISTA-PLV/coinductive/tree/2b2ee94fa326e96835e021556ee52fcf6a235ea2>, Apr. 2026.
- [4] L. M. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer, “The Lean Theorem Prover (System Description),” in *Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings*, 2015, pp. 378–388. doi: 10.1007/978-3-319-21401-6_26.
- [5] A. Keizer, “Implementing a definitional (co)datatype package in Lean 4, based on quotients of polynomial functors,” Master's thesis, <https://eprints.illc.uva.nl/id/eprint/2239/1/MoL-2023-03.text.pdf>, 2023.
- [6] J. Avigad, M. Carneiro, and S. Hudon, “Data Types as Quotients of Polynomial Functors,” in *10th International Conference on Interactive Theorem Proving (ITP 2019)*, J. Harrison, J. O’Leary, and A. Tolmach, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 141. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 6:1–6:19. doi: 10.4230/LIPIcs.ITP.2019.6.
- [7] The Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: <https://homotopytypetheory.org/book>, 2013.
- [8] S. Awodey, “Cambridge Lectures on Categorical Logic and Type Theory.” <https://awodey.github.io/cltt/>, Mar. 2026.
- [9] nLab authors, “polynomial functor.” Jan. 2026.
- [10] J. Lambek, “A Fixpoint Theorem for complete Categories.,” *Mathematische Zeitschrift*, vol. 103, pp. 151–161, 1968, [Online]. Available: <http://eudml.org/doc/170906>
- [11] J. Adámek, “Free algebras and automata realizations in the language of categories,” *Commentationes Mathematicae Universitatis Carolinae*, vol. 15, no. 4, pp. 589–602, 1974, [Online]. Available: <http://eudml.org/doc/16649>
- [12] M. P. Fiore, “A Coinduction Principle for Recursive Data Types Based on Bisimulation,” *Information and Computation*, vol. 127, no. 2, pp. 186–198, 1996, doi: <https://doi.org/10.1006/inco.1996.0058>.
- [13] Z. Yang and N. Wu, “Fantastic Morphisms and Where to Find Them,” in *Mathematics of Program Construction*, E. Komendantskaya, Ed., Cham: Springer International Publishing, 2022, pp. 222–267.
- [14] T. Uustalu and V. Vene, “Primitive (Co)Recursion and Course-of-Value (Co)Iteration, Categorically,” *Informatica*, vol. 10, p. , 1999.
- [15] R. Hinze, N. Wu, and J. Gibbons, “Unifying structured recursion schemes,” in *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming*, in ICFP '13. Boston, Massachusetts, USA: Association for Computing Machinery, 2013, pp. 209–220. doi: 10.1145/2500365.2500578.

- [16] B. Goodspeed, “Proof Driven Development,” *CoRR*, 2015, [Online]. Available: <http://arxiv.org/abs/1512.02102>
- [17] J. Kim, Y. Nam, and C.-K. Hur, “Coco: Corecursion with Compositional Heterogeneous Productivity,” *Proc. ACM Program. Lang.*, vol. 10, no. POPL, Jan. 2026, doi: 10.1145/3776733.
- [18] The Agda Team, “Universe Levels — Agda 2.8.0 Documentation.” Accessed: May 03, 2026. [Online]. Available: <https://agda.readthedocs.io/en/v2.8.0-r3/language/universe-levels.html>
- [19] nLab authors, “Kleisli category.” May 2026.
- [20] Y. Zakowski, P. He, C.-K. Hur, and S. Zdancewic, “An equational theory for weak bisimulation via generalized parameterized coinduction,” in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, in CPP 2020. New Orleans, LA, USA: Association for Computing Machinery, 2020, pp. 71–84. doi: 10.1145/3372885.3373813.
- [21] D. Pous, “Coinduction All the Way Up,” in *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science*, in LICS '16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 307–316. doi: 10.1145/2933575.2934564.
- [22] X. Leroy, “Formal verification of a realistic compiler,” *Commun. ACM*, vol. 52, no. 7, pp. 107–115, July 2009, doi: 10.1145/1538788.1538814.
- [23] M. Fehr, T. Grosser, L. Cicolini, and L. Stefanescu, “Coinductive.” <https://github.com/opencompl/veir/tree/ca1dd404e93e93ac2eb94ab7a36da578bd20b6f1>, Apr. 2026.

6 Appendices

6.A Project import graph

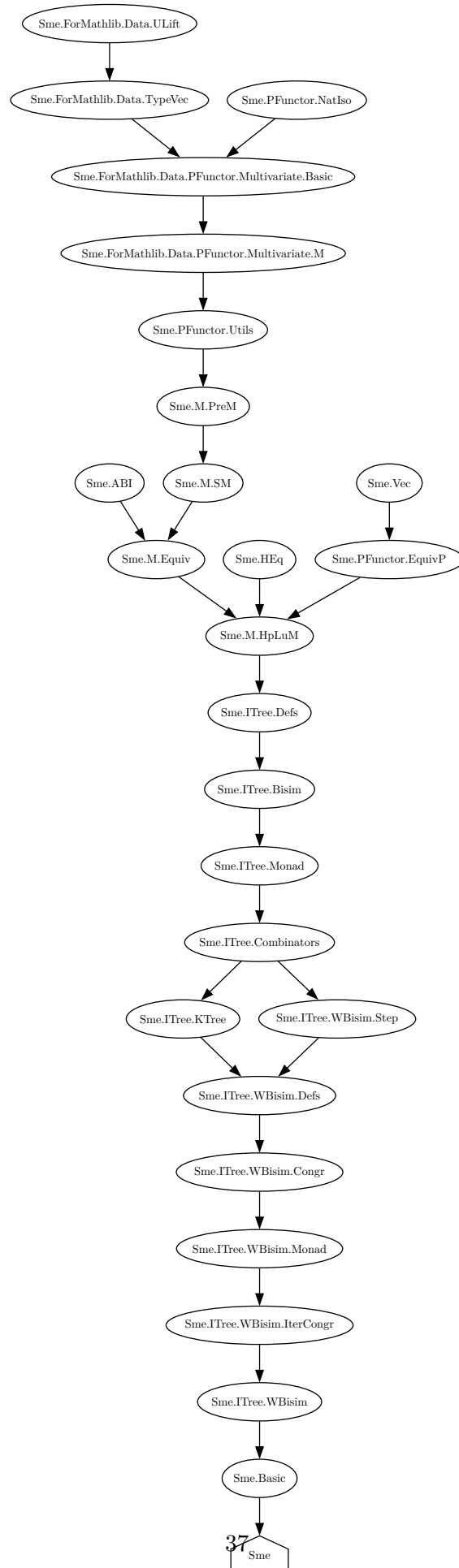


Figure 30: Import graph of the lean component of the project as of 2026-04-04

²⁵TODO CITE

7 Proposal

See proposal overleaf

Efficient coinductives through state-machine corecursors

Supervisor: Alex Keizer

Marking supervisor: Jamie Vicary

Director of studies: Russell Moore

Word count: 993

1 Work to be undertaken

Previous work has shown how coinductive types can be encoded in Lean as quotients of polynomial functors (QPFs)[1]. This is done through progressive approximation (Section 4.1.2). For example accessing the n index of a stream takes $\mathcal{O}(n^2)$ time. This becomes an issue when using Lean as a general purpose programming language. An alternative approach is using a state-machine based encoding (Section 4.1.1) of cofixed points. With these we get nice computational behaviour of the compiled code. In Figure 1 we can see in blue the performance from the current implementation, versus in orange the performance of a prototype implementation the state-machine based version.

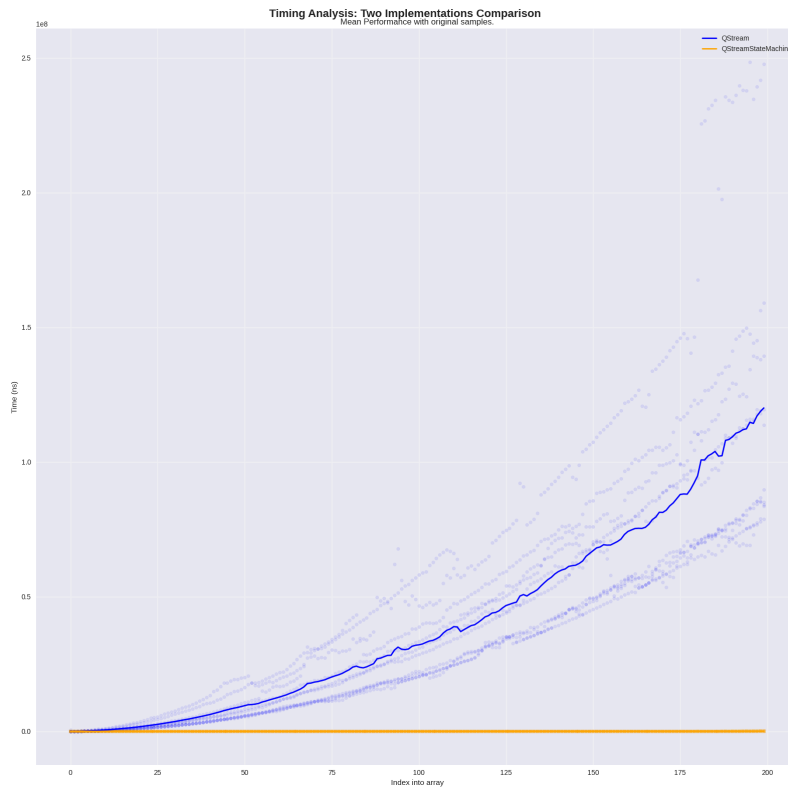


Figure 1: Graph plotting current performance v possible gains

2 Core project

This project will be implementing the state-machine encoding (Section 4.1.1) of coinductive types, and formalising the equivalence between these two embedding (Section 4.3). I will start with the special example of equivalence between `Streams` in the two representations, then building up to the general cofix structure.

After this I will be implementing a coinductive monad allowing us to encode non-termination as an effect using the state-machine representation. This will be an instance of the two representations.

3 Starting point

I have worked with QPFs on the meta-programming side for an internship between my Part Ia and Part Ib. During this I learnt of the basics of polynomial functors, and `TypeVecs`. This means I am aware of what the underlying structures are when it comes to the raw implementation. I have also done a feasibility assessment of the project by seeing how the current polynomials respond to universe levels. This led to me making 2 pull requests (#28095, #28279) to mathlib on `TypeVec` in preparation for my project. I also tried to merge (#28112) an implementation of commutable `Shrink` to mathlib that later got reverted when it was found to be inconsistent.

There are more minor refactoring pull requests towards the mathlib repository which don't change any behaviour but in general all of these can be *found on this link*. I include these for completeness and transparency.

4 Substance

Section 4.1.1, Section 4.3 and Section 4.4 all discuss structures that will be implemented throughout the project. Section 4.1.2 and Section 4.2 both are implemented in Avigard et al [1].

4.1 The M type

The M type is the name given to the terminal coalgebra of polynomial functors; the possibly infinitely deep trees. This means the implementations must have both: A corecursor `corec : {α : Type} → (f : α → F α) → α → MF`, and a destructor `dest : MF → F(MF)`. We have two encodings of the M type.

4.1.1 State-machine encoding

The state-machine encoding is the naïve way to implement the terminal coalgebra. Given some polynomial functor F , the state-machine encoding is given by: some type $\alpha : \text{Type}$, a function $f : \alpha \rightarrow F\alpha$, and some witness $a : \alpha$. Then you quotient over bisimilarity, thereby only allowing direct usage of `dest : Sme F → F(Sme F)`. The baseline implementation of this should be straight-forward.

4.1.2 Progressive approximation encoding

The other way is to generate them by progressive approximation where earlier trees must “agree” with the later ones. Agreement is given by them being the same up to the previous depth as seen in Figure 2.

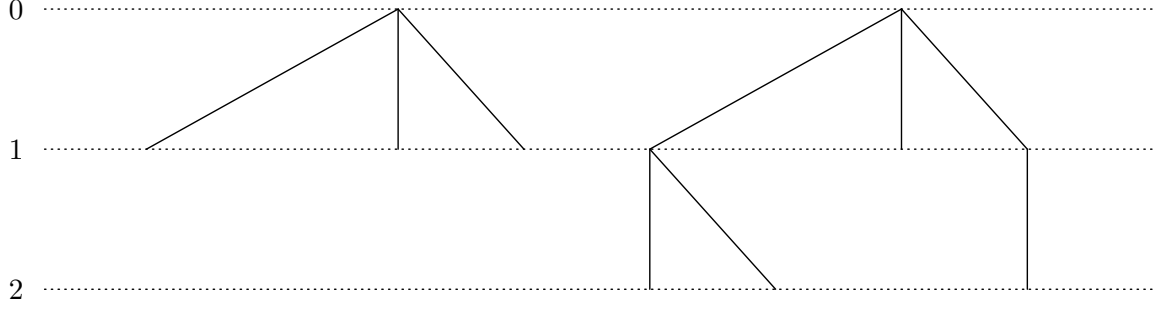


Figure 2: Agreement of two trees of height 1 and 2

4.2 The Cofix type

Cofix is the terminal coalgebra in QPFs, These are simply a quotient over M types lifting the quotient from the source QPF.

4.3 The equivalence

The equivalence between the current implementation of **Cofix** and M and the state-machine representation is the core of the project. The functions in both directions are given by $f : \text{corec}_{\text{Pae}} \text{dest}_{\text{Sme}}$ and $f^{-1} : \text{corec}_{\text{Sme}} \text{dest}_{\text{Pae}}$. The difficulty comes from proving $f \circ f^{-1} = \mathbb{1}$ and $f^{-1} \circ f = \mathbb{1}$. The proof of these equalities will be proven using bisimilarity, and other parts of the theory currently established for M types. Simpler versions of this equivalence also exist for the simpler representation.

4.4 The non-termination monad

The non-termination monad is a simple example of a coinductive. This will be useful for testing the state-machine representation's performance. The structure of the monad as a coinductive has two constructors as seen in this pseudocode:

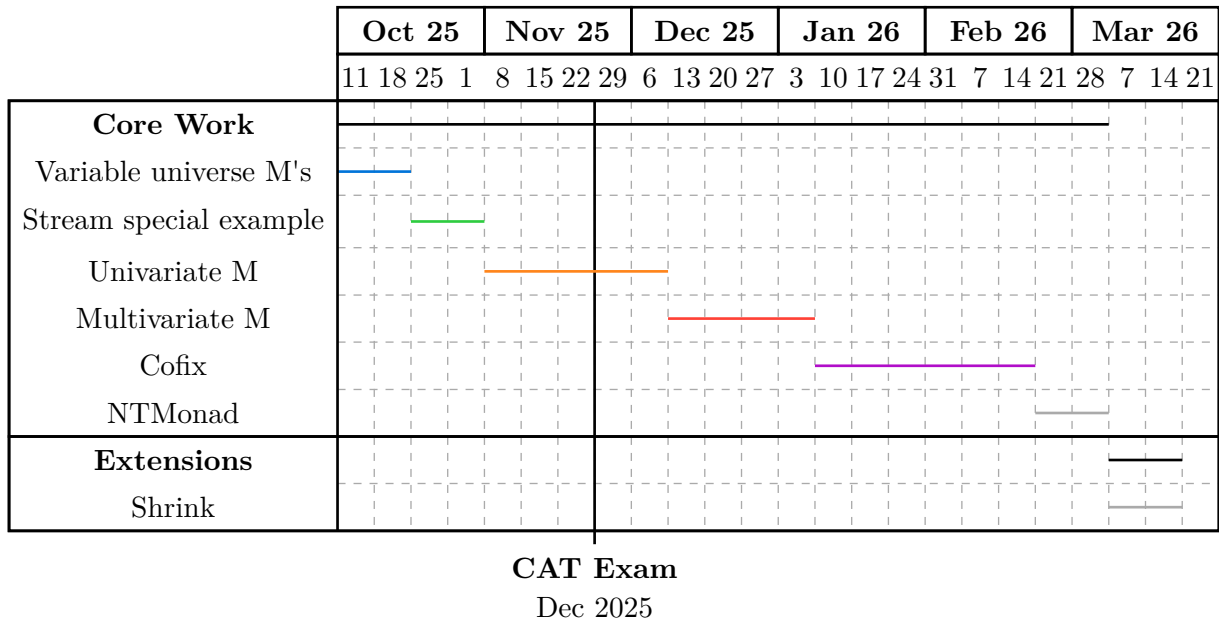
```
coinductive NTMonad (A : Type)
| val : A -> NTMonad A
| tau : NTMonad A -> NTMonad A
```

5 Evaluation

This projects success can be broken down into a few disjoint criteria.

1. If the equivalence (Section 4.3) has been proven,
2. how close to the theorised performance we get with the state-machine encoding,
3. and the ability to construct the example structures.

6 Core timeline



The plan for work would be divided into a few different stages.

6.1 Variable universe Ms (2025-10-11 2w 2025-10-24)

To begin with I have to make the corecursor universe polymorphic. As stated in Section 3 I have most of this code written. I will attempt to get this merged into mathlib, but if this review process takes too long I will be working on my own branch.

6.2 Stream special example (2025-10-25 2w 2025-11-7)

The first step would be implementing `Stream` under the two representations. Then I will familiarise myself with using bisimilarity to formalise the equivalence. This is a first step to just understand how these features work.

6.3 Univariate M (2025-11-8 2w 2025-11-21 + (CAT exam) + 2025-12-05 3w 2025-12-23)

Implementing the univariate `M` type is the natural next step from this. This will be the natural next step in difficulty. The work should lay the groundwork for the next proofs.

6.4 Multivariate M (2025-12-26 4w 2026-01-23)

The next step will be generalising the univariate implementation, the same structure of the proof should be the same for the multivariate case. The main difficulty here comes from working with `TypeVecs`. These are quite difficult to reason about.

6.5 Cofix (2026-01-24 6w 2026-03-06)

Finally, the `Cofix` type has to be implemented. This will be the most challenging part as I will have to work with quotients.

6.6 Implementing the NTMonad (2026-03-07 2w 2026-03-20)

This will be a demonstration of all of the previous work.

7 Extensions

7.1 Shrinking the representations (2026-03-21 2w 2026-04-04)

This involves finding a sound way allowing the current theory to use the efficient representation. The soundness of this will be difficult to reason about as it requires working with the Lean code generator. Therefore I leave this as an extension.

8 Resources

Access to the restricted side of the lab is needed for working with the greater team.

Bibliography

- [1] J. Avigad, M. Carneiro, and S. Hudon, “Data Types as Quotients of Polynomial Functors,” in *10th International Conference on Interactive Theorem Proving (ITP 2019)*, J. Harrison, J. O’Leary, and A. Tolmach, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 141. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 6:1–6:19. doi: 10.4230/LIPIcs.ITP.2019.6.